TEC | Tecnológico de Costa Rica

Entregable Proyecto 1

Escuela de Ingeniería en Computadores

CE1108 – Compiladores e intérpretes

Profesor:
Leiton Jiménez Jason

Estudiante:
Araya Porras Luis Diego – 2021086423
Piedra Montero Joseph - 2023047830
Somarribas Montero Isaac - 2020125516

Fecha de entrega:
Miércoles 14 de octubre del 2026

Semestre II, 2026

1. Introducción y filosofía del lenguaje

Nova (del latín nova, “nueva”) es un lenguaje de programación imperativo, estructurado, de tipado estático, fuerte y explícito, de formato libre y compilado. Su diseño toma ideas de Python (anotaciones de tipo con :, operadores $and/or/not$, la cadena $elif$, comentarios con #) y de Java/C (bloques con llaves, sentencias terminadas con ;, tipos primitivos de tamaño fijo, arreglos de tamaño conocido), pero no replica a ninguno de los dos: combina ambas herencias resolviendo de forma distinta los puntos que hacen difícil compilar a Python. Sus palabras reservadas están en latín (apartado b de la sección 1).

- Nombre del lenguaje: Nova
- Extensión de archivos fuente: $.nv$
- Extensión de los archivos objeto: $.nvo$
- Compilador: $novac$

Destino de compilación: Nova compilará hacia el procesador de una arquitectura propia desarrollada por el grupo de CE4301, cuyo ISA incluye, entre otras cosas, una unidad criptográfica Feistel4 y una bóveda de llaves. Esa decisión no es cosmética: fija el ancho de los datos en 32 bits, obliga a que el lenguaje exponga las operaciones de bits que usa el algoritmo de cifrado y descarta el punto flotante.

a. Decisiones de diseño y su justificación

#	Problema observado	Decisión tomada en Nova
1	Python usa tipado dinámico	Tipado estático: todo lleva su tipo declarado y se verifica en compilación. El tipo de una variable es inmutable

2	Conversiones implícitas	No existen, y como hay un solo tipo numérico tampoco hacen falta conversiones explícitas
3	Truthiness: cualquier valor es condición	La condición de si, sin, dum y per debe ser $veritas$
4	Indentación significativa	Formato libre: bloques con { } y sentencias con ;. El lexer no lleva estado de indentación
5	Funciones con más de una entrada	De 0 a 255 parámetros, cada uno con su tipo. Los argumentos pueden ser expresiones.
6	¿$si/aliter$ o también sin?	Se incluye sin: es azúcar sintáctica que el parser convierte en si anidado, así que no agrega ni un nodo al AST

Otras decisiones que abaratan el compilador: llaves obligatorias en todo el cuerpo (eliminan el $dangling\ else$), la asignación es sentencia y no expresión (hace imposible si $(x = 1)$), todo dato no inicializado vale cero (elimina el análisis de flujo), los globales solo pueden ser $constans$, no hay sobrecarga de funciones ni memoria dinámica, y hay un único mecanismo de importación con espacio de nombres.

b. Palabras reservadas en latín

Las palabras reservadas de Nova están tomadas del latín. La decisión es deliberada y su costo de implementación es nulo: las palabras son entradas de una tabla que el analizador léxico consulta al reconocer un identificador, para decidir si emite el token de la palabra reservada o TK_IDENT. La gramática, el AST, el analizador semántico y el generador de código no ven palabras sino tokens, así que ninguna de esas fases cambia. A cambio, el lenguaje queda claramente diferenciado de Python y Java desde la primera línea de código.

Correspondencia con los términos habituales, como referencia de lectura:

Nova (latín)	Equivalente usual	Nova (latín)	Equivalente usual
$functio$	$func\ /\ def$	et	and
$varia$	var	aut	or
$constans$	$const\ /\ final$	non	not
$privatum$	$private$	$verum$	$true$
$importa$	$import$	$falsum$	$false$
ut	as	$integer$	int
$redde$	$return$	$veritas$	$bool$
si	if	$textus$	$string$
sin	$elif\ /\ else\ if$	$nihil$	$void$
$aliter$	$else$	$longitudo$	$len\ /\ length$
dum	$while$	$scribe$	$print$
per	for	dic	$println$
$rumpe$	$break$	$principale$	$main$
$perge$	$continue$		

Todas son palabras latinas reales: sin significa “pero si”, $aliter$ “de otro modo”, dum “mientras”, $redde$ “devuelve”, $rumpe$ “rompe”, $perge$ “sigue” y $nihil$ “nada”. Se escriben sin macrones ni diacríticos, porque los identificadores y las palabras reservadas son ASCII.

Los mensajes de error del compilador, en cambio, se emiten en español: el latín es para escribir programas, no para depurarlos.

2. Convenciones léxicas

a. Formato

Texto $UTF-8$. Identificadores y palabras reservadas en ASCII; se admite $UTF-8$ dentro de cadenas y comentarios. Formato libre: el espacio en blanco solo separa tokens. Sensible a mayúsculas.

El lexer aplica la regla del bocado más largo: ante >= produce TK_MAYOR_IGUAL y no TK_MAYOR seguido de TK_ASIGNA.

b. Comentarios

Forma	Sintaxis	Descripción
De línea	$\#\ldots hasta\ fin\ de\ línea$	Termina con el salto de línea
De bloque	#{ ... }#	Varias líneas. No se anida: el primer }# lo cierra.

Ambos los elimina el analizador léxico, por lo que nunca llegan al sintáctico. Un bloque sin cerrar produce $E103$

$\#\ Comentario\ de\ línea$
$\#\{\ Comentario\ de\ bloque\colon puede\ contener\ llaves\ \{\quad\}\ y\ punto\ y\ coma\ ;\ \}\#$
$varia\ x\ :\ integer = 10;\ \#\ al\ final\ de\ una\ sentencia$

c. Identificadores y literales

$identificador$ ::= ($letra$ | "_") { $letra$ | $digito$ | "_" } ($máx. 255\ caracteres$)
$ENTERO$ ::= $[0-9]+$ | $0x[0-9A-Fa-f]+$
$CADENA$ ::= " ([^"\\\n] | $ESCAPE$) * "
$ESCAPE$ ::= \\ (n | t | \\ | ")

Hay literales hexadecimales ($0x1F,\ 0xDEADBEEF$) porque las llaves y las máscaras de bits del algoritmo de cifrado se escriben así de forma natural; agregarlos al autómata cuesta dos estados. No hay literales reales (no existe el tipo, ver sección 3), ni binarios, ni de carácter. Las cadenas no ocupan varias líneas.

$scribe$, dic, $longitudo$ y las cuatro intrínsecas del apartado b de la sección 3 no son palabras reservadas: son identificadores predefinidos en el ámbito global.

d. Tabla de tokens

Son 23 palabras reservadas y 36 símbolos.

Categoría	Lexemas	Tokens
Reservadas de declaración	$functio - varia - constans - privatum - importa - ut - redde$	$TK_FUNCTIO$ TK_VARIA $TK_CONSTANS$ $TK_PRIVATUM$ $TK_IMPORTA$ TK_UT TK_REDDE
Reservadas de control	$si - sin - aliter - dum - per - rumpe - perge$	TK_SI TK_SIN TK_ALITER TK_DUM TK_PER TK_RUMPE TK_PERGE
Reservadas de tipo	$integer - veritas - textus - nihil$	$TK_TIPO_INTEGER$ $TK_TIPO_VERITAS$ TK_TIPO_TEXTUS TK_TIPO_NIHIL
Operadores lógicos	$et - aut - non$	TK_ET TK_AUT TK_NON
Literales booleanos	$verum - falsum$	TK_VERUM TK_FALSUM
Operadores aritméticos	$+ \ - * / \%$	TK_MAS TK_MENOS TK_POR TK_ENTRE TK_MODULO
Operadores relacionales	$== ! = < <= > >=$	TK_IGUAL $TK_DIFERENTE$ TK_MENOR TK_MENOR_IGUAL

Operadores de asignación	$= \ += \ -= \ *= \ /= \ \%=$	TK_MAYOR TK_MAYOR_IGUAL TK_ASIGNA TK_ASIGNA_MAS TK_ASIGNA_MENOS TK_ASIGNA_POR TK_ASIGNA_ENTRE TK_ASIGNA_MODULO
Delimitadores	$(\ ) \ \{ \ \} \ [\ ] \ ; \ ,$	TK_PAR_IZQ TK_PAR_DER TK_LLAVE_IZQ TK_LLAVE_DER TK_COR_IZQ TK_COR_DER TK_PUNTO_COMA TK_COMA
Símbolos especiales	$: \ . \ -> \ @$	TK_DOS_PUNTOS $TK_PUNTO \ TK_FLECHA$ TK_ARROBA
Identificadores	$total, matriz_a, _tmp$	TK_IDENT
Constantes	$42, 0xFF,$ "$hola$"	TK_ENTERO TK_CADENA
Fin de archivo	(marca virtual)	TK_EOF

Operadores de bits. Van aparte porque uno de ellos es la barra vertical. Todos mapean a una sola instrucción de la ALU del procesador de destino.

&	$TK_AMPERSAND$	AND bit a bit
|	TK_BARRA	OR bit a bit
^	TK_XOR	XOR bit a bit

~	TK_NOT	NOT bit a bit
<<	TK_DESP_IZQ	Desplazamiento a la izquierda
>>	TK_DESP_DER	Desplazamiento a la derecha, con relleno de signo
>>>	TK_DESP_LOG	Desplazamiento a la derecha, con relleno de ceros

El XOR es indispensable: es la operación central de la red Feistel. El desplazamiento lógico >>> existe porque la rotación de 32 bits del algoritmo, que se escribe $(x << r)$ | $\big(x >>> (32 - r)\big)$, necesita relleno con ceros y no con signo.
Las cinco asignaciones compuestas se expanden en el parser a la forma simple: $a \mathrel{+}= b$ da $a = a + b$, de modo que el resto del compilador solo conoce $=$.
Cada token lleva su lexema, línea y columna, que son los que aparecen en los mensajes de error.
Reservadas para versiones futuras (usarlas como identificadores es $E208$):
$classis\ structura\ enumeratio\ novum\ dele\ elige\ casus\ fac\ tempta\ cape\ nullum\ hoc\ stativum\ publicum\ modulus\ ex\ littera\ relatio\ compara\ in\ realis\ naturalis$.

3. Sistema de tipos

Tipo	Tamaño	Dominio	Valor por defecto
$integer$	4 bytes	Entero con signo de 32 bits, complemento a 2	0
$veritas$	4 bytes	$verum, falsum$	$falsum$
$textus$	Descriptor	Cadena inmutable de solo lectura	""

$nihil$	–	Solo como tipo de retorno de una función	–

Cada valor ocupa una palabra de 32 bits, que es el ancho de los registros del procesador destino. $integer$ es el único tipo numérico: no hay punto flotante, porque el procesador de destino no tiene unidad de punto flotante y un $realis$ no tendría a qué instrucción traducirse. Esto elimina de un solo golpe las conversiones, la promoción de tipos y la duplicación de reglas en cada operador.

Reglas:

- Estático: El tipo de toda expresión se determina en compilación.
- Fuerte: Sin conversiones implícitas ni explícitas, al haber un solo tipo numérico, no hay ninguna conversión que hacer.
- Explícito: Sin inferencia.
- Inmutable: La variable conserva su tipo.
- Los operandos de un operador binario deben ser del mismo tipo; el resultado de un aritmético o de bits es de ese tipo y el de un relacional es veritas.
- $/$ es división entera truncada hacia cero; $\%$ es el residuo.
- $veritas$ no es un entero: No hay conversión con $integer$ en ninguna dirección.
- Todo dato sin inicializar vale su valor por defecto: No hay contenido indefinido.
- La aritmética de $integer$ es modular de 32 bits (envolvente), que es justo lo que exige la suma módulo 2^{32} de la función de ronda de Feistel4.

a. Funciones predefinidas

Función	Firma	Descripción
$longitudo$	$longitudo(a) -> integer$	Tamaño de la primera dimensión de un arreglo, o largo de un $textus$

$scribe$	$scribe(e1, e2, \ldots) -> nihil$	Imprime los argumentos separados por un espacio
dic	$dic(e1, e2, \ldots) -> nihil$	Igual, con salto de línea al final

$scribe$ y dic son las únicas construcciones variádicas; el usuario no puede definir funciones variádicas. El compilador las traduce, según el tipo de cada argumento, a llamadas monomórficas del soporte de ejecución, sin despacho dinámico.
Sobre la salida. El procesador no tiene consola: los datos entran y salen de la memoria mediante las herramientas de carga y extracción de archivos del grupo de $CE4301$. Por eso $scribe$ y dic se compilan a escrituras sobre una zona de salida mapeada en memoria. Si el ISA final no la incluye, ambas funciones se eliminan del lenguaje y los programas comunican sus resultados dejándolos en memoria, que es el modelo que usan de todas formas las herramientas de validación.

b. Funciones intrínsecas de la unidad criptográfica

El procesador de destino incluye una unidad Feistel4 y una bóveda de llaves a las que solo se llega con instrucciones específicas. Nova las expone como intrínsecas: identificadores predefinidos que el compilador traduce a una instrucción del ISA, no a una llamada a función. No son palabras reservadas y no se pueden redefinir.

$ingredere(clave : integer) -> veritas$
$exi(\ \ ) -> nihil$
$custodi(ranura : integer, k0 : integer, k1 : integer, k2 : integer, k3 : integer) -> nihil$
$feistel(bloque : integer[\ \ ], ranura : integer, ronda : integer) -> nihil$

Intrínseca	Descripción

$ingredere$	Autentica el programa ante la bóveda; devuelve $verum$ si la clave es correcta. Sin autenticación previa, el resto de las intrínsecas aborta
exi	Cierra la sesión autenticada
$custodi$	Guarda en la ranura indicada de la bóveda (0 a 3) una llave de 128 bits, dada como cuatro subllaves de 32 bits
$feistel$	Ejecuta una ronda de Feistel4 sobre $bloque[0]$ (L) y $bloque[1]$ (R), con la subllave de la ronda indicada de esa ranura. Modifica el arreglo en sitio

$feistel$ recibe un arreglo y no dos variables sueltas porque los arreglos viajan por referencia: así una sola instrucción produce el nuevo par (L, R) sin que el lenguaje necesite retorno múltiple. Cifrar un bloque es encadenar cuatro rondas, y descifrarlo es recorrerlas en orden inverso.

```
functio cifrar(bloque : integer[], ranura : integer) -> nihil {
    per (varia r : integer = 0; r < 4; r += 1) {
        feistel(bloque, ranura, r);
    }
}
functio descifrar(bloque : integer[], ranura : integer) -> nihil {
    per (varia r : integer = 3; r >= 0; r -= 1) {
        feistel(bloque, ranura, r);
    }
}
```

Las llaves nunca pasan por una variable del programa: $custodi$ las deposita en la bóveda y a partir de ahí solo $feistel$ puede leerlas, que es exactamente la restricción de aislamiento que exige el proyecto de arquitectura. Si $ranura$ o $ronda$

son constantes fuera del rango 0 a 3, es error $E319$; si no lo son, se verifica en ejecución.

4. Definición de variables

$varia < identificador > : < tipo > [= < inicializador >];$
$constans < identificador > : < tipo > = < expresion_constante >;$

La anotación de tipo es obligatoria. El inicializador es opcional: si se omite, la variable toma el valor por defecto de su tipo. Redeclarar en el mismo ámbito es $E302$ y asignar a una $constans$ es $E311$.

$varia\ x : integer = 5;$
$varia\ activo : veritas = verum;$
$varia\ nombre : textus = "Nova";$
$varia\ contador : integer;$ $\#\ queda\ en\ 0$
$constans\ MAX : integer = 3;$
$varia\ tabla : integer[MAX][MAX];$ $\#\ tamaño\ dado\ por\ una\ constante$
$varia\ v : integer[4] = \{10, 20, 30, 40\};$
$x = x + 1;$ $\#\ asignación\ simple$
$x \mathrel{+}= 1;$ $\#\ asignación\ compuesta$
$tabla[1][2] = x * 2;$

Declaraciones globales. Fuera de las funciones solo se admiten $constans$ y $functio$; una $varia$ global es $E320$. Sin estado global mutable no hay que definir el orden de inicialización entre módulos ni permitir escrituras entre archivos.

$constans\ VERSION : textus = "1.0";$
$privatum\ constans\ SEMILLA : integer = 7;$ $\#\ visible\ solo\ dentro\ del\ módulo$

5. Expresiones y operadores

De mayor a menor precedencia:

Nivel	Operadores	Asociatividad
1	$(\ldots)\ f(\ldots)\ a[i]\ m.f$	Izquierda
2	$-\ non\ \sim\ (unarios)$	Derecha
3	$*\ /\ \%$	Izquierda
4	$+\ -$	Izquierda
5	$<<\ >>\ >>>$	Izquierda
6	$<\ <=\ >\ >=$	Izquierda
7	$==\ !=$	Izquierda
8	$\&$	Izquierda
9	^	Izquierda
10	|	Izquierda
11	et	Izquierda
12	aut	Izquierda

Doce niveles, uno por método del analizador descendente recursivo. La asignación no aparece porque no es un operador sino una sentencia; por eso $a = b = c$ y $si\ (x = 1)$ son errores de sintaxis y no errores silenciosos de lógica.

Los desplazamientos quedan por debajo de los aditivos, a diferencia de C, para que $a + b << 2$ signifique $(a + b) << 2$, que es lo que casi siempre se quiere. Los operadores de bits quedan por debajo de los relacionales, también a diferencia de C, para que $x\ \&\ 1 == 0$ signifique $x\ \&\ (1 == 0)$ ... que no es lo deseable, así que se toma la convención opuesta a C y se ubican &, ^ y | por encima de et y aut pero por debajo de la igualdad: $x\ \&\ 1 == 0$ es un error de tipos $(E303)$, lo que obliga a escribir $(x\ \&\ 1) == 0$ y elimina el problema en lugar de esconderlo.

Tipos. $+\ -\ *\ /\ \%$ y los operadores de bits $\&\ |\ ^\ \sim\ <<\ >>\ >>>$ operan sobre $integer$ y devuelven $integer$; los relacionales devuelven $veritas$; $==$ y $!=$ exigen el mismo tipo primitivo en ambos lados y sobre $textus$ comparan contenido; et, aut

y non solo operan sobre $veritas$. et y aut evalúan en corto circuito, lo que permite escribir $si\ (i < longitudo(v)\ et\ v[i] > 0)$. Los arreglos no admiten ningún operador: para copiarlos hay que recorrerlos.

El desplazamiento con una cantidad negativa o mayor o igual que 32 produce un resultado indefinido, tal como en C y en Java; es responsabilidad del programa acotarla.

En ejecución. La división entre cero y el índice fuera de rango abortan con archivo y línea (la verificación de rango está siempre activa); el desbordamiento de $integer$ es aritmética modular de 32 bits, documentada y usada a propósito por el algoritmo de cifrado.

6. Estructuras de control

Toda sentencia simple termina en ;. Los cuerpos son siempre bloques con llaves, aunque tengan una sola sentencia, lo que elimina la ambigüedad del $dangling\ else$. Las condiciones van entre paréntesis y deben ser $veritas$ $(E306)$.

$si\ (< cond >)\ \{\ \ldots\ \}\ sin\ (< cond >)\ \{\ \ldots\ \}\ aliter\ \{\ \ldots\ \}$ # sin: $cero\ o\ más$
$dum\ (< cond >)\ \{\ \ldots\ \}$
$per\ (< inicializacion >\ ;\ < cond >\ ;\ < paso >)\ \{\ \ldots\ \}$

En el per, la inicialización es una declaración $varia$ sin ; o una asignación, y el paso es una asignación; las tres partes son obligatorias. Si la inicialización declara una variable, su ámbito es el ciclo. No existen $++$ ni $--$: se usa $i\ +=\ 1$.

$rumpe$ y $perge$ actúan sobre el ciclo más interno ($E310$ fuera de un ciclo). $redde <$ $expr >$; en funciones con tipo y $redde$; en las $nihil$. Toda ruta de una función no $nihil$ debe terminar en $redde$ $(E309)$.

$functio\ clasificar(x : integer) -> nihil\ \{$

```
    si (x > 0) {
        dic("positivo");
    }sin (x < 0) {
        dic("negativo");
    } aliter {
        dic("cero");
    }
}
per (varia i : integer = 0; i < 10; i+= 1) {
    dic(i);
}
```

No hay ciclo con condición al final ni selección múltiple (*fac* y *elige* quedan reservadas para el futuro): *dum e si/sin* expresan lo mismo sin agregar reglas a la gramática ni nodos al AST.

7. Funciones y llamadas

```
[ @anotacion ]
[ privatum ] functio < nombre > ( < p1 > : < t1 >, < p2 > : < t2 >, ... ) ->
    < tipo_ret > {
        < sentencias >
}
```

- De 0 a 255 parámetros, cada uno con su tipo explícito.
- El tipo de retorno es obligatorio, después de $->$; $nihil$ si no retorna valor.
- No hay sobrecarga $(E302)$, ni parámetros por omisión, ni funciones anidadas, ni clausuras: el nombre identifica la firma de forma única.
- La recursión directa y mutua está permitida: una primera pasada recolecta las firmas del archivo, así que una función puede llamar a otra declarada más abajo.
- Todo programa ejecutable tiene exactamente una $functio\ principale(\ ) ->$ $integer$ ($E405$ si falta, $E406$ si está duplicada); el valor retornado es el código de salida.

Paso de parámetros. $integer$ y $veritas$ van por valor; $textus$ por referencia de solo lectura; los arreglos por referencia, de modo que modificarlos dentro de la función sí afecta al llamador. Como los arreglos no pueden retornarse, el patrón para "devolver" una matriz es recibirla como parámetro de salida.

Llamadas. $nombre(args)$ o $modulo.nombre(args)$ para otro módulo. Los argumentos pueden ser expresiones arbitrarias, incluidas otras llamadas. La cantidad $(E304)$ y el tipo $(E305)$ deben coincidir exactamente, sin promoción automática. Se evalúan de izquierda a derecha. Descartar el valor retornado genera la advertencia $W005$.

$functio\ potencia(base : integer,\ exponente : integer) -> integer\ \{$

$varia\ resultado : integer = 1;$

$per\ (varia\ i : integer = 0; i < exponente; i += 1)\ \{$

$resultado *= base;$

$\}$

$redde\ resultado;$

}

$varia\ r : integer = suma(3,4);$	$\#\ uso\ como\ expresión$
$saludar(\quad);$	$\#\ uso\ como\ sentencia$
$varia\ q : integer = suma(suma(1,2), longitudo(v) - 1);$	$\#\ expresiones\ y\ llamadas\ anidadas$
$matriz.imprimir(m);$	$\#\ llamada\ calificada$

Como no hay clausuras ni cantidad variable de argumentos definida por el usuario, el tamaño del marco de pila de cada función se conoce en compilación.

8. Ámbito de bloque

El ámbito se delimita con llaves { }, nunca con indentación ni con begin/end. Hay tres niveles: de módulo (funciones y constantes del archivo), de función (parámetros y cuerpo) y de bloque (cualquier { }, incluidos los de si, dum y per).

1. Declarar antes de usar: un identificador es visible desde su declaración hasta el } que cierra su bloque. Las funciones y constantes del módulo son visibles en todo el archivo gracias a la pasada de recolección de firmas.
2. Redeclarar en el mismo ámbito es E302; ocultar un nombre de un ámbito exterior es válido, pero produce la advertencia W002.
3. La variable declarada en el per pertenece al ámbito del ciclo.

4. La tabla de símbolos es una pila de tablas hash, una por ámbito abierto: abrir un bloque apila, cerrarlo desapila y buscar es recorrer la pila desde la cima.

```
functio demo(p : integer) -> nihil {
    varia a : integer = p;
    si (a > 0) {
        varia b : integer = a * 2;          # b: solo dentro del si
        dic(b);
    }
    # dic(b);                               # E301: 'b' no existe en este ámbito
    per (varia i : integer = 0; i < 3; i += 1) {
    varia a : integer = i;                  # W002: oculta a la 'a' exterior
    dic(a);
    }
    # dic(i);           # E301: 'i' murió con el ciclo
}
```

9. Arreglos de dos dimensiones

Declaración. varia $<$ nombre $>$: $<$ tipo $>$ [$<$ filas $>$][$<$ columnas $>$];, donde filas y columnas son expresiones constantes enteras positivas evaluables en compilación (E318 si no lo son). Máximo dos dimensiones. Los elementos se guardan en orden por filas: m[i][j] está en $\text{base} + (\text{i} * \text{M} + \text{j}) * \text{tamaño(T)}$.

Inicialización. De dos formas, y ambas son válidas. Con lista de valores constantes, cuyas dimensiones deben coincidir exactamente con el tipo declarado (E317). O bien declarando el arreglo y llenándolo después con ciclos: al omitirse la lista, todos los elementos quedan en cero y el programa los asigna cuando quiera.

Acceso. El índice es integer (E307) y empieza en 0. $m[i][j]$ es el elemento y $m[i]$ la fila completa, usable como argumento de $longitudo$ o de un parámetro $T[]$. $longitudo(m)$ da las filas y $longitudo(m[0])$ las columnas. La verificación de rango en ejecución está siempre activa; con índice constante fuera de rango se detecta en compilación (E308).

Paso a funciones. Los arreglos viajan por referencia mediante un descriptor con la dirección base y el tamaño de cada dimensión, lo que permite declarar parámetros con dimensiones abiertas ($integer[]$, i$nteger[][]$) y consultar el tamaño real con $longitudo$. El compilador verifica el tipo de elemento y la cantidad de dimensiones (E314); comparar los tamaños concretos es responsabilidad del programa. Las dimensiones abiertas solo se permiten en parámetros.

```
# Forma 1: declaración con lista de valores
varia kernel : integer[3][3] = {{0, 1, 0},
                                {1, -4, 1},
                                {0, 1, 0}};

# Forma 2: declaracion vacia (queda en ceros) y llenado en un ciclo
varia matriz : integer[3][3];
per (varia i : integer = 0; i < 3; i += 1) {
    per (varia j : integer = 0; j < 3; j += 1) {
        matriz[i][j] = i * 3 + j;
```

```
    }
}

functio sumar_matrices(a : integer[][], b : integer[][], destino : integer[][]) -> nihil {
    per (varia i : integer = 0; i < longitudo(a); i += 1) {
        per (varia j : integer = 0; j < longitudo(a[0]); j += 1) {
            destino[i][j] = a[i][j] + b[i][j];
        }
    }
}
```

10. Anotaciones (pragmas)

Metadatos dirigidos al compilador, escritos con @ antes de una declaración de función.

Anotación	Efecto
@cauda	(cauda = cola) Verifica que todas las llamadas recursivas de la función estén en posición de cola y las convierte en un salto al inicio, reasignando los parámetros. Si alguna no lo está, es E315.

$@obsoletum("mensaje")$	(obsoletum = obsoleto) Cada llamada produce la advertencia W006 con ese mensaje.

Una anotación desconocida no detiene la compilación: produce W004 y se descarta.

Posición de cola: la llamada es lo último que hace la función antes de retornar, es decir, aparece como redde f(...);, o como f(...); seguida de redde; en una función nihil.

```
@cauda

functio contar(n : integer, acumulado : integer) -> integer {

    si (n == 0) {

        redde acumulado;

    }

    redde contar(n − 1, acumulado + n);      # posición de cola: se compila como salto

}
```

11. Módulos e importación

Un archivo es un módulo, y su nombre es el del archivo sin $.nv$ (debe ser un identificador válido; si no, E409). Toda declaración global se exporta por omisión; $privatum$ la hace privada del módulo.

La estrategia elegida es una sola: importación por módulo con espacio de nombres.

Forma	Sintaxis	Acceso
Simple	*importa "matematica";*	*matematica.suma(3, 5)*
Con alias	*importa "matematica" ut mat;*	*mat.suma(3, 5)*

Se descartaron la inclusión textual (duplica código, rompe la compilación separada y reporta los errores en el archivo equivocado) y la importación selectiva (mete nombres ajenos en el ámbito del módulo). Con acceso calificado, una colisión de nombres entre dos módulos es imposible por construcción.

Reglas:

5. Las importaciones van al inicio del archivo, antes de cualquier declaración (E206).
6. La cadena es la ruta lógica sin extensión y admite subdirectorios (importa "util/matriz"; da el espacio matriz).
7. La importación no es transitiva; e importar dos veces el mismo espacio de nombres es E402.

Los módulos se buscan primero en el directorio del archivo que importa y luego en cada directorio dado con $-I$ (E404 si no aparece). El compilador construye el grafo de importaciones y lo recorre en profundidad para detectar ciclos (E403) y obtener el orden de compilación. Cada símbolo se decora como $modulo\$simbolo$, y la verificación de tipos entre archivos es completa: llamar $matematica.suma(verum, falsum)$ cuando recibe dos integer se detecta al compilar, no al ejecutar.

12. Compilación y enlace

a. Fases

#	Fase	Entrada	Salida	Errores

1	Análisis Léxico	Texto fuente	Flujo de tokens	E1xx
2	Análisis Sintáctico	Tokens	AST	E2xx
3	Análisis Semántico	AST	AST decorado + tabla de símbolos	E3xx
4	Código intermedio	AST decorado	Código de tres direcciones, ensamblador VLIW y ejecutable	E4xx
5	Asignación de registros y planificación	Código intermedio	Código del destino	-
6	Emisión y enlace	Código de los módulos	Archivo binario ejecutable	E4xx

El plegado de constantes ocurre dentro de la fase 3, porque el compilador ya debe evaluar expresiones constantes para conocer los tamaños de los arreglos. No se promete ninguna otra optimización.

Las fases 4, 5 y 6 se implementan en un solo paso del compilador, descrito en la sección 22. De la 5 está hecha la selección de instrucciones y no la planificación; de la 6, la emisión y no el enlace.

b. Código intermedio y destino

La fase 4 produce código de tres direcciones sobre una máquina abstracta con registros virtuales ilimitados. El repertorio completo es:

t = a <op> b	*t = <op> a*	*t = a*	*t = #c*
t = &x	*t = arr[i]*	*arr[i] = t*	*t = longitudo a, dim n*
check i, n	*param t*	*t = call f, n*	*call f, n*
tailcall f, n	*goto L*	*L:*	*ret t*
ret	*ifFalse t goto L*	*ifTrue t goto L*	*functio f: / fin f*
authw #w, t	*vault.login*	*vault.logout*	*vault.store #kv, #w, t*
feistel t, #kv, #ronda			

Las cinco últimas corresponden a las intrínsecas de la sección 3 apartado b y son las únicas operaciones que el generador traduce a instrucciones de la unidad criptográfica. Esta representación es independiente del procesador: Es el punto donde se separa todo lo que ya está construido de lo que depende del ISA.

c. Varios archivos

Modo por omisión: programa completo. $novac\ principal.nv\ -o$ principal sigue las importaciones, compila todos los módulos alcanzables en una corrida y produce el ejecutable, con todas las tablas de símbolos en memoria al mismo tiempo.

Modo separado. Con $-c$, cada módulo produce un $.nvo$ en formato de texto con su interfaz exportada y su código intermedio:

#modulo matriz

#interfaz

functio matriz$limitar(integer, integer, integer) -> integer

functio matriz$imprimir(integer[][]) -> nihil

#requiere

(ninguno)

#codigo

... codigo de tres direcciones ...

Enlazar es leer los objetos, verificar que cada símbolo requerido exista con la firma correcta (E407), concatenar el código y traducirlo.

d. Invocación del compilador

$$novac\ [opciones]\ <archivo_fuente>\ [archivos\ adicionales...]\ [-o\ <salida>]$$

Opción	Descripción
-o <archivo>	Nombre de salida. Si se omite, el del primer fuente sin extensión (o con .nvo si se usa -c)
-c	Solo compila, no enlaza: genera el archivo objeto .nvo
-I <directorio>	Agrega un directorio de búsqueda de módulos. Puede repetirse
-v	Modo detallado: imprime cada fase y el archivo procesado
--tokens	Imprime la tabla de tokens: lexema, token, línea y columna
-t	Imprime el árbol sintáctico (AST) con sangría

--sym	Imprime la tabla de símbolos de cada ámbito
--ir	Imprime el código de tres direcciones
-s	Imprime el ensamblador VLIW de CERBERO en texto
-Wall / -Werror	Activa todas las advertencias / las trata como errores
--max-errors <n>	Máximo de errores antes de abortar (25 por omisión)
--version, -h	Versión y ayuda de uso

Las cinco banderas de volcado (--tokens, -t, --sym, --ir, -s) imprimen estructuras que el compilador construye de todos modos y permiten probar cada fase por separado. Códigos de salida: 0 éxito, 1 errores de compilación, 2 error de uso, 3 error de entrada o salida.

```
novac programa.nv                          # binario con nombre por defecto
novac programa.nv -o salida                # nombre de salida explicito
novac programa.nv -v -t                    # fases y arbol sintactico
novac --tokens programa.nv                 # solo los tokens
novac -c modulo.nv -o modulo.nvo           # solo compilar, sin enlazar
novac principal.nv -Wall -o convolucion    # programa completo
novac -c matriz.nv                         # compilacion separada
novac -c estadistica.nv
novac principal.nv matriz.nvo estadistica.nvo -o convolucion
novac principal.nv -I ./lib -o convolucion
./convolucion
```

e. Resolución de errores entre archivos

Se reportan todos juntos al final; no hay recuperación y no se emite ejecutable.

Símbolo importado no encontrado:

principal.nv:14:20: error[E401]: el modulo 'estadistica' no exporta 'promedio'

14 | dic(estadistica.promedio(m));

*| ^^^^^^^^^^^*

= nota: 'estadistica' exporta: maximo, minimo, suma_total

= ayuda: quizás quiso escribir 'suma_total ', o el símbolo está marcado 'privatum'

Recuperación: si el compilador intenta usar un símbolo que no existe en el módulo importado, reporta el error una sola vez y continúa analizando el resto del programa. De esta forma se evitan mensajes repetidos causados por el mismo problema. Al finalizar el análisis, la compilación se detiene y no se genera el ejecutable.

Conflicto de nombres entre módulos importados: como los símbolos se usan mediante el nombre del módulo, dos módulos pueden tener funciones o constantes con el mismo nombre sin causar conflicto. Por ejemplo, matriz.suma() y estadistica.suma() pueden coexistir. El conflicto aparece únicamente cuando dos importaciones terminan usando el mismo nombre de módulo o espacio de nombres.

principal.nv:3:8: error[E402]: el espacio de nombres 'matriz' ya fue importado

3 | importa "img/matriz";

*| ^^^^^^^^^^^*

= nota: importacion previa de "util/matriz" en principal.nv:2:8

= ayuda: use un alias: importa "img/matriz" ut imgmat;

Recuperación: prevalece la primera importación, la segunda se marca como inválida y la compilación continúa para reportar el resto de los errores.

Importación circular:

error[E403]: importacion circular detectada

= ciclo: matriz.nv -> estadistica.nv -> matriz.nv

= ayuda: mueva las definiciones compartidas a un tercer modulo (comun.nv)

Recuperación: ninguna. El ciclo impide construir el orden de compilación, así que el compilador aborta de inmediato.

Otros: E404 módulo no encontrado, E405 falta principale, E406 principale duplicada, E407 firma incompatible entre objetos, E408 símbolo privado, E409 nombre de módulo inválido.

13. Manejo de errores

a. Formato de los diagnósticos

<archivo>:<linea>:<columna>: <severidad>[<codigo>]: <mensaje>

<n> | <linea de codigo fuente>

| <marca ^^^ bajo el lexema culpable>

= nota: <informacion adicional>

= ayuda: <sugerencia de correccion>

Severidades: $error$ (impide generar código) y $advertencia$ (no lo impide). La línea y la columna son las del primer carácter del lexema donde se detectó el problema; el lexer las guarda en cada token, así que están disponibles en todas las fases. Al final se imprime un resumen: $compilacion\ abortada: 3\ errores,\ 1\ advertencia.$

b. Errores léxicos (E1XX)

Código	Situación	Mensaje
E101	Carácter no reconocido	Carácter no válido '$' fuera de una cadena
E102	Cadena sin cerrar	Cadena sin comilla de cierre antes del fin de línea
E103	Comentario de bloque sin cerrar	Comentario de bloque sin cerrar; falta '}#'
E104	Escape inválido	Secuencia de escape desconocida '\q'
E105	Número mal formado	Literal numérico mal formado: '0x'
E106	Entero fuera de rango	El literal no cabe en un integer de 32 bits
E107	Identificador demasiado largo	Identificador de más de 255 caracteres

Recuperación: se descarta el carácter o lexema ofensivo y se continúa desde el siguiente carácter, de modo que un $ perdido no oculta el resto de los errores. Una cadena sin cerrar se asume cerrada al final de la línea.

c. Errores sintácticos (E2XX)

Código	Situación	Mensaje
E201	Token inesperado	Se esperaba ')' pero se encontró '{'
E202	Falta ;	Falta ';' al final de la sentencia
E203	Llave sin pareja	'{' abierto en la línea 12 nunca se cierra
E204	Falta anotación de tipo	Falta ': tipo' en la declaración de 'x'
E205	aliter/sin sin si	'aliter' sin un 'si' correspondiente
E206	Construcción fuera de lugar	Las importaciones deben ir antes de cualquier declaración
E207	Cuerpo sin llaves	El cuerpo de 'dum' debe ir entre llaves
E208	Palabra reservada futura	'classis' está reservada para versiones futuras
E209	Falta tipo de retorno	Falta '-> tipo' en la declaración de la función 'f'

Recuperación: modo pánico con tokens de sincronización. Se descartan tokens hasta encontrar ;, } o el inicio de una declaración (functio, varia, constans, importa) y se retoma desde ahí. Para el ; faltante se aplica además corrección a nivel de frase: se inserta virtualmente, se reporta E202 y el análisis continúa sin descartar nada. Tras --max-errors errores el compilador aborta.

d. Errores semánticos (E3XX)

Código	Situación	Código	Situación
E301	Identificador no declarado	E312	Tipo de retorno incorrecto

E302	Redeclaración en el mismo ámbito	E313	Se invoca algo que no es función
E303	Tipos incompatibles	E314	Dimensiones de arreglo incompatibles
E304	Cantidad de argumentos	E315	@cauda incumplida
E305	Tipo de un argumento	E316	División entre la constante cero
E306	Condición no booleana	E317	Lista de inicialización de tamaño incorrecto
E307	Índice no entero	E318	Tamaño de arreglo no constante
E308	Índice constante fuera de rango	E319	Argumento inválido de una intrínseca
E309	Falta redde en alguna ruta	E320	Declaración global que no es constans
E310	rumpe/perge fuera de ciclo	E321	Uso de nihil fuera del retorno
E311	Asignación a una constante		

```
prueba.nv:12:29: error[E303]: no se puede asignar 'textus' a una variable de tipo 'integer'
12 | varia total : integer = "cero";
   |                         ^^^^^^^^
= nota: 'textus' e 'integer' son tipos distintos y no existe conversión entre ellos
```

Recuperación: propagación de un tipo especial error. Cuando una expresión falla la verificación, su nodo recibe ese tipo, y toda operación que reciba un operando error

se considera válida y también produce error, sin volver a reportar. Así un error real produce un solo mensaje en lugar de una cascada.

e. Advertencias y errores de ejecución

Advertencias: W001 variable declarada y nunca usada, W002 una declaración oculta a otra de un ámbito exterior, W003 código inalcanzable, W004 anotación desconocida, W005 se descarta el valor retornado, W006 uso de una función @obsoletum y W007 el marco de una función o los datos globales no caben en el espacio que el ISA les da.

En ejecución solo existen tres errores, porque no hay punteros nulos, ni memoria dinámica, ni conversiones que puedan fallar, ni excepciones:

Situación	Mensaje	Código de salida
Índice fuera de rango	error en ejecucion: indice 7 fuera de rango (tamano 5) en prueba.nv:14	134
División entre cero	error en ejecucion: division entre cero en prueba.nv:22	136
Bóveda sin autenticar	error en ejecucion: acceso no autorizado a la boveda en prueba.nv:31	137

En CERBERO no hay consola, así que el programa generado no imprime el mensaje: deja el código en R6 y ejecuta halt. El mensaje es el que produciría una implementación alojada sobre un sistema operativo.

14. Gramática formal (EBNF)

{ x } es cero o más repeticiones, [x] es opcional, x | y es alternativa, "x" es un terminal y MAYUSCULAS es un token del analizador léxico.

```
programa          ::= { importacion } { declaracion_global } TK_EOF
importacion       ::= "importa" CADENA [ "ut" IDENT ] ";"

declaracion_global  ::= { anotacion } [ "privatum" ] ( decl_func | decl_const )
anotacion         ::= "@" IDENT [ "(" CADENA ")" ]
decl_func         ::= "functio" IDENT "(" [ parametros ] ")" "->" tipo bloque
parametros        ::= parametro { "," parametro }
parametro         ::= IDENT ":" tipo_param

tipo              ::= tipo_base { "[" expresion "]" }
tipo_param        ::= tipo_base { "[" [ expresion ] "]" }
tipo_base         ::= "integer" | "veritas" | "textus" | "nihil"

bloque            ::= "{" { sentencia } "}"
sentencia         ::= decl_var | decl_const | sent_asignacion | sent_if |
sent_while
                    | sent_for | sent_return | sent_break | sent_continue
                    | sent_llamada | bloque

decl_var          ::= "varia" IDENT ":" tipo [ "=" inicializador ] ";"
decl_const        ::= "constans" IDENT ":" tipo "=" expresion ";"
inicializador     ::= expresion | lista_valores
lista_valores     ::= "{" [ inicializador { "," inicializador } ] "}"
```

```
sent_asignacion      ::= asignacion ";"
asignacion           ::= designador op_asig expresion
designador           ::= IDENT { "[" expresion "]" }
op_asig              ::= "=" | "+=" | "-=" | "*=" | "/=" | "%="

sent_if              ::= "si" "(" expresion ")" bloque
                         { "sin" "(" expresion ")" bloque } [ "aliter" bloque ]
sent_while           ::= "dum" "(" expresion ")" bloque
sent_for             ::= "per" "(" init_for ";" expresion ";" asignacion ")" bloque
init_for             ::= "varia" IDENT ":" tipo "=" expresion | asignacion

sent_return          ::= "redde" [ expresion ] ";"
sent_break           ::= "rumpe" ";"
sent_continue        ::= "perge" ";"
sent_llamada         ::= llamada ";"

expresion            ::= exp_or
exp_or               ::= exp_and { "aut" exp_and }
exp_and              ::= exp_bitor { "et" exp_bitor }
exp_bitor            ::= exp_bitxor { "|" exp_bitxor }
exp_bitxor           ::= exp_bitand { "^" exp_bitand }
exp_bitand           ::= exp_igualdad { "&" exp_igualdad }
exp_igualdad         ::= exp_relacional { ( "==" | "!=" ) exp_relacional }
exp_relacional       ::= exp_desplaz { ( "<" | "<=" | ">" | ">=" ) exp_desplaz }
exp_desplaz          ::= exp_aditiva { ( "<<" | ">>" | ">>>" ) exp_aditiva }
exp_aditiva          ::= exp_multiplicativa { ( "+" | "-" ) exp_multiplicativa }
exp_multiplicativa   ::= exp_unaria { ( "*" | "/" | "%" ) exp_unaria }
exp_unaria           ::= [ "-" | "non" | "~" ] exp_primaria
exp_primaria         ::= literal | "(" expresion ")" | referencia
referencia           ::= IDENT [ "." IDENT ]
```

```
                    ( "(" [ argumentos ] ")" | { "[" expresion "]" } )
llamada             ::= IDENT [ "." IDENT ] "(" [ argumentos ] ")"
argumentos          ::= expresion { "," expresion }
literal             ::= ENTERO | CADENA | "verum" | "falsum"
```

La gramática es LL(1) tal como está escrita, así que se implementa con un analizador descendente recursivo: un método por regla, sin tablas ni generadores. La única decisión que podría exigir dos tokens de anticipación (distinguir f(...) de v[...]) se resolvió factorizando por la izquierda en referencia. No hay ambigüedad de dangling else porque los cuerpos son siempre bloque. nihil aparece en tipo_base por simplicidad, pero el analizador semántico lo rechaza (E321) fuera del tipo de retorno.

15. Programas de ejemplo

Los tres programas van con su equivalente en Python para evidenciar que producen el mismo resultado.

a. Multiplicación de matrices con recursividad de cola

```
# multiplicacion.nv
# La matriz resultado ingresa por parametro, porque en Nova los arreglos se
pasan
# por referencia y no pueden retornarse.

functio imprimir(m : integer[][]) -> nihil {
    per (varia i : integer = 0; i < longitudo(m); i += 1) {
```

```
        per (varia j : integer = 0; j < longitudo(m[0]); j += 1) {
            scribe(m[i][j]);
        }
        dic("");
    }
}

@cauda
functio multiplicar(a : integer[][], b : integer[][], c : integer[][],
                    i : integer, j : integer, k : integer, acumulado : integer) -> nihil {
    si (i >= longitudo(a)) {
        redde;  # terminaron todas las filas
    }
    si (j >= longitudo(b[0])) {
        multiplicar(a, b, c, i + 1, 0, 0, 0);  # siguiente fila
        redde;
    }
    si (k >= longitudo(b)) {
        c[i][j] = acumulado;  # celda terminada
        multiplicar(a, b, c, i, j + 1, 0, 0);  # siguiente columna
        redde;
    }
    multiplicar(a, b, c, i, j, k + 1, acumulado + a[i][k] * b[k][j]);
    redde;
}

functio principale() -> integer {
    varia a : integer[2][3] = {{1, 2, 3},
                               {4, 5, 6}};
    varia b : integer[3][2] = {{7, 8},
```

```
                        {9, 10},
                        {11, 12}};
    varia c : integer[2][2];

    si (longitudo(a[0]) != longitudo(b)) {
        dic("error: dimensiones incompatibles");
        redde 1;
    }

    multiplicar(a, b, c, 0, 0, 0, 0);
    imprimir(c);
    redde 0;
}
```

```
# multiplicacion.py -- equivalente
def imprimir(m):
    for i in range(len(m)):
        for j in range(len(m[0])):
            print(m[i][j], end=" ")
        print()

def multiplicar(a, b, c, i, j, k, acumulado):
    if i >= len(a):
        return
    if j >= len(b[0]):
        return multiplicar(a, b, c, i + 1, 0, 0, 0)
    if k >= len(b):
        c[i][j] = acumulado
        return multiplicar(a, b, c, i, j + 1, 0, 0)
    return multiplicar(a, b, c, i, j, k + 1, acumulado + a[i][k] * b[k][j])
```

```
def main():
    a = [[1, 2, 3], [4, 5, 6]]
    b = [[7, 8], [9, 10], [11, 12]]
    c = [[0, 0], [0, 0]]
    if len(a[0]) != len(b):
        print("error: dimensiones incompatibles")
        return 1
    multiplicar(a, b, c, 0, 0, 0, 0)
    imprimir(c)
    return 0

main()
```

Salida de ambos: 58 64 / 139 154. Python no optimiza la recursión de cola y necesitaría sys.setrecursionlimit para matrices grandes; Nova la convierte en un salto gracias a @cauda, así que la pila se mantiene constante.

b. Merge sort iterativo

```
# mergesort.nv
# Merge sort ascendente (bottom-up), sin recursion. El arreglo auxiliar ingresa
# por parametro porque Nova no tiene memoria dinamica.

functio minimo(x : integer, y : integer) -> integer {
    si (x < y) {
        redde x;
    }
    redde y;
}
```

```
11
12 functio fusionar(a : integer[], aux : integer[], izq : integer, medio : integer,
13   der : integer) -> nihil {
14     varia i : integer = izq;
15     varia j : integer = medio;
16
17     per (varia t : integer = izq; t < der; t += 1) {
18         si (i < medio et (j >= der aut a[i] <= a[j])) {
19             aux[t] = a[i];
20             i += 1;
21         } aliter {
22             aux[t] = a[j];
23             j += 1;
24         }
25     }
26     per (varia t : integer = izq; t < der; t += 1) {
27         a[t] = aux[t];
28     }
29 }
30
31 functio merge_sort(a : integer[], aux : integer[]) -> nihil {
32     varia n : integer = longitudo(a);
33     varia ancho : integer = 1;
34
35     dum (ancho < n) {
36         varia inicio : integer = 0;
37         dum (inicio < n) {
38             varia medio : integer = minimo(inicio + ancho, n);
39             varia der   : integer = minimo(inicio + 2 * ancho, n);
40             fusionar(a, aux, inicio, medio, der);
41             inicio += 2 * ancho;
42         }
43         ancho *= 2;
44     }
45 }
46
47 functio principale() -> integer {
48     varia datos : integer[8] = {38, 27, 43, 3, 9, 82, 10, 1};
```

```
    varia aux   : integer[8];

    merge_sort(datos, aux);
    per (varia i : integer = 0; i < longitudo(datos); i += 1) {
        scribe(datos[i]);
    }
    dic("");
    redde 0;
}

```

Equivalente en Python

```python
# mergesort.py -- equivalente
def fusionar(a, aux, izq, medio, der):
    i, j = izq, medio
    for t in range(izq, der):
        if i < medio and (j >= der or a[i] <= a[j]):
            aux[t] = a[i]
            i += 1
        else:
            aux[t] = a[j]
            j += 1
    for t in range(izq, der):
        a[t] = aux[t]

def merge_sort(a, aux):
    n = len(a)
    ancho = 1
    while ancho < n:
        inicio = 0
        while inicio < n:
            medio = min(inicio + ancho, n)
            der = min(inicio + 2 * ancho, n)
            fusionar(a, aux, inicio, medio, der)
            inicio += 2 * ancho
        ancho *= 2
```

```
25
26 def main():
27     datos = [38, 27, 43, 3, 9, 82, 10, 1]
28     aux = [0] * len(datos)
29     merge_sort(datos, aux)
30     for x in datos:
31         print(x, end=" ")
32     print()
33     return 0
34
35 main()
```

La salida de ambos programas es la siguiente: **1 3 9 10 27 38 43 82**. En la versión de Nova se declara `minimo` porque `min` no es una función predefinida del lenguaje como sucede en Python.

c. Convolución 2D con importación de dos archivos

El siguiente algoritmo evidencia el uso de per, si, llamadas a métodos, expresiones como argumentos y la importación de dos archivos. Se eligió la convolución de matrices porque es la primera prueba prevista para el compilador.

```
 1 # matriz.nv -- archivo 1
 2 functio limitar(v : integer, minimo : integer, maximo : integer) -> integer {
 3     si (v < minimo) {
 4         redde minimo;
 5     } sin (v > maximo) {
 6         redde maximo;
 7     } aliter {
 8         redde v;
 9     }
10 }
11
12 functio imprimir(m : integer[][]) -> nihil {
```

```
    per (varia i : integer = 0; i < longitudo(m); i += 1) {
        per (varia j : integer = 0; j < longitudo(m[0]); j += 1) {
            scribe(m[i][j]);
        }
        dic("");
    }
}

functio es_impar(n : integer) -> veritas {
    redde n % 2 == 1;
}
```

```
# estadistica.nv -- archivo 2
# 'privatum' hace que esta funcion no se exporte, es interna del modulo.
privatum functio total(m : integer[][]) -> integer {
    varia suma : integer = 0;
    per (varia i : integer = 0; i < longitudo(m); i += 1) {
        per (varia j : integer = 0; j < longitudo(m[0]); j += 1) {
            suma += m[i][j];
        }
    }
    redde suma;
}

functio promedio(m : integer[][]) -> integer {
    varia celdas : integer = longitudo(m) * longitudo(m[0]);
    redde total(m) / celdas;   # division entera: no hay punto flotante
}

functio maximo(m : integer[][]) -> integer {
    varia mayor : integer = m[0][0];
    per (varia i : integer = 0; i < longitudo(m); i += 1) {
        per (varia j : integer = 0; j < longitudo(m[0]); j += 1) {
            si (m[i][j] > mayor) {
                mayor = m[i][j];
            }
        }
```

```
    }
    redde mayor;
}
```

```
# convolucion.nv -- programa principal
importa "matriz";                           # dos archivos importados,
importa "estadistica";                      # cada uno con su espacio de nombres

constans LADO : integer = 5;

functio convolucionar(imagen : integer[][], kernel : integer[][], salida :
  integer[][]) -> nihil {
    varia alto  : integer = longitudo(imagen);
    varia ancho : integer = longitudo(imagen[0]);
    varia radio : integer = longitudo(kernel) / 2;     # division entera:
                                                       # ambos son integer

    per (varia y : integer = 0; y < alto; y += 1) {
        per (varia x : integer = 0; x < ancho; x += 1) {
            varia suma : integer = 0;
            per (varia dy : integer = -radio; dy <= radio; dy += 1) {
                per (varia dx : integer = -radio; dx <= radio; dx += 1) {
                    # llamadas a otro modulo con expresiones como argumentos
                    varia fy : integer = matriz.limitar(y + dy, 0, alto - 1);
                    varia fx : integer = matriz.limitar(x + dx, 0, ancho - 1);
                    suma += imagen[fy][fx] * kernel[dy + radio][dx + radio];
                }
            }
            salida[y][x] = suma;
        }
    }
}

functio principale() -> integer {
    varia imagen : integer[LADO][LADO] = {{10, 10, 10, 10, 10},
                                          {10, 50, 50, 50, 10},
                                          {10, 50, 90, 50, 10},
```

```
                                        {10, 50, 50, 50, 10},
                                        {10, 10, 10, 10, 10}};

    varia kernel : integer[3][3] = {{ 0, -1,  0},
                                    {-1,  5, -1},
                                    { 0, -1,  0}};

    varia salida : integer[LADO][LADO];

    si (non matriz.es_impar(longitudo(kernel))) {
        dic("error: el kernel debe tener lado impar");
        redde 1;
    }

    convolucionar(imagen, kernel, salida);
    dic("resultado:");
    matriz.imprimir(salida);
    dic("promedio =", estadistica.promedio(salida));
    dic("maximo   =", estadistica.maximo(salida));
    redde 0;
}
```

Equivalente en Python

```
# matriz.py -- equivalente del archivo 1
def limitar(v, minimo, maximo):
    if v < minimo:
        return minimo
    elif v > maximo:
        return maximo
    else:
        return v

def imprimir(m):
    for i in range(len(m)):
        for j in range(len(m[0])):
            print(m[i][j], end=" ")
```

```
        print()

def es_impar(n):
    return n % 2 == 1
```

```
# estadistica.py -- equivalente del archivo 2
def _total(m):
    suma = 0
    for i in range(len(m)):
        for j in range(len(m[0])):
            suma += m[i][j]
    return suma

def promedio(m):
    celdas = len(m) * len(m[0])
    return _total(m) // celdas

def maximo(m):
    mayor = m[0][0]
    for i in range(len(m)):
        for j in range(len(m[0])):
            if m[i][j] > mayor:
                mayor = m[i][j]
    return mayor
```

```
# convolucion.py -- equivalente del programa principal
import matriz
import estadistica

LADO = 5

def convolucionar(imagen, kernel, salida):
    alto, ancho = len(imagen), len(imagen[0])
    radio = len(kernel) // 2
    for y in range(alto):
        for x in range(ancho):
            suma = 0
```

```
            for dy in range(-radio, radio + 1):
                for dx in range(-radio, radio + 1):
                    fy = matriz.limitar(y + dy, 0, alto - 1)
                    fx = matriz.limitar(x + dx, 0, ancho - 1)
                    suma += imagen[fy][fx] * kernel[dy + radio][dx + radio]
            salida[y][x] = suma

def main():
    imagen = [[10, 10, 10, 10, 10],
              [10, 50, 50, 50, 10],
              [10, 50, 90, 50, 10],
              [10, 50, 50, 50, 10],
              [10, 10, 10, 10, 10]]
    kernel = [[0, -1, 0], [-1, 5, -1], [0, -1, 0]]
    salida = [[0] * LADO for _ in range(LADO)]
    if not matriz.es_impar(len(kernel)):
        print("error: el kernel debe tener lado impar")
        return 1
    convolucionar(imagen, kernel, salida)
    print("resultado:")
    matriz.imprimir(salida)
    print("promedio =", estadistica.promedio(salida))
    print("maximo   =", estadistica.maximo(salida))
    return 0

main()
```

Salida esperada en ambos casos

```
resultado:
10 -30 -30 -30 10
-30 130 50 130 -30
-30 50 250 50 -30
-30 130 50 130 -30
10 -30 -30 -30 10
promedio = 26
maximo = 250
```

Compilación

```
novac convolucion.nv -Wall -o convolucion     # programa completo

novac -c matriz.nv                            # o compilacion separada
novac -c estadistica.nv
novac convolucion.nv matriz.nvo estadistica.nvo -o convolucion
./convolucion
```

16. Alcance de la versión 1.0

Para ser realistas con lo que el grupo se compromete a implementar, no forman parte del lenguaje: el punto flotante, porque el procesador destino no tiene unidad de punto flotante; objetos y herencia; memoria dinámica y recolección de basura; retorno de arreglos y concatenación de cadenas (se usa el patrón de parámetro de salida); el tipo char; literales binarios y exponenciales; variables globales mutables; sobrecarga de funciones y parámetros por omisión; excepciones; arreglos de más de dos dimensiones; punteros; genéricos, clausuras y funciones anidadas; selección múltiple, ciclo con condición al final, ++/-- y el operador ternario; optimizaciones más allá del plegado de constantes; y entrada de datos por teclado o archivo.

17. Analizador léxico

a. Principio de diseño

Las 23 palabras reservadas no tienen un autómata propio. El lexer reconoce un único patrón de identificador y luego consulta una tabla para decidir si emite el token de la palabra reservada o TK_IDENT:

IDENT -> [A-Za-z_][A-Za-z0-9_]*

Esto evita 23 autómatas que compiten entre sí y es lo que hace que agregar o quitar una palabra reservada cueste una línea de tabla. Las palabras reservadas para versiones futuras se consultan en la misma pasada y producen el error E208.

Como el lenguaje es de formato libre, el espacio en blanco solo separa tokens y se descarta. Los comentarios también se descartan: nunca llegan al analizador sintáctico.

b. Tabla de expresiones regulares

Las 64 reglas se dan en notación POSIX extendida. \ escapa un metacarácter

#	Categoría	Lexema	Token	Expresión regular
1	Declaración	functio	TK_FUNCTIO	functio
2	Declaración	varia	TK_VARIA	varia
3	Declaración	constans	TK_CONSTANS	constans
4	Declaración	privatum	TK_PRIVATUM	privatum
5	Declaración	importa	TK_IMPORTA	importa
6	Declaración	ut	TK_UT	ut
7	Declaración	redde	TK_REDDE	redde
8	Control	si	TK_SI	si
9	Control	sin	TK_SIN	sin
10	Control	aliter	TK_ALITER	aliter
11	Control	dum	TK_DUM	dum
12	Control	per	TK_PER	per
13	Control	rumpe	TK_RUMPE	rumpe
14	Control	perge	TK_PERGE	perge

15	Tipo	integer	TK_TIPO_INTEGER	integer
16	Tipo	veritas	TK_TIPO_VERITAS	veritas
17	Tipo	textus	TK_TIPO_TEXTUS	textus
18	Tipo	nihil	TK_TIPO_NIHIL	nihil
19	Lógico	et	TK_ET	et
20	Lógico	aut	TK_AUT	aut
21	Lógico	non	TK_NON	non
22	Booleano	verum	TK_VERUM	verum
23	Booleano	falsum	TK_FALSUM	falsum
24	Aritmético	+	TK_MAS	\+
25	Aritmético	-	TK_MENOS	-
26	Aritmético	*	TK_POR	*
27	Aritmético	/	TK_ENTRE	/
28	Aritmético	%	TK_MODULO	%
29	Relacional	==	TK_IGUAL	==
30	Relacional	!=	TK_DIFERENTE	!=
31	Relacional	<	TK_MENOR	<
32	Relacional	<=	TK_MENOR_IGUAL	<=
33	Relacional	>	TK_MAYOR	>
34	Relacional	>=	TK_MAYOR_IGUAL	>=
35	Asignación	=	TK_ASIGNA	=
36	Asignación	+=	TK_ASIGNA_MAS	\+=
37	Asignación	-=	TK_ASIGNA_MENOS	-=
38	Asignación	*=	TK_ASIGNA_POR	*=
39	Asignación	/=	TK_ASIGNA_ENTRE	/=
40	Asignación	%=	TK_ASIGNA_MODULO	%=
41	Delimitador	(	TK_PAR_IZQ	\(
42	Delimitador	)	TK_PAR_DER	\)
43	Delimitador	{	TK_LLAVE_IZQ	\{

44	Delimitador	}	TK_LLAVE_DER	\}
45	Delimitador	[	TK_COR_IZQ	\[
46	Delimitador	]	TK_COR_DER	\]
47	Delimitador	;	TK_PUNTO_COMA	;
48	Delimitador	,	TK_COMA	,
49	Especial	:	TK_DOS_PUNTOS	:
50	Especial	.	TK_PUNTO	\.
51	Especial	->	TK_FLECHA	->
52	Especial	@	TK_ARROBA	@
53	Bits	&	TK_AMPERSAND	&
54	Bits	|	TK_BARRA	|
55	Bits	^	TK_XOR	\^
56	Bits	~	TK_NOT	~
57	Bits	<<	TK_DESP_IZQ	<<
58	Bits	>>	TK_DESP_DER	>>
59	Bits	>>>	TK_DESP_LOG	>>>
60	Identificador	(variable)	TK_IDENT	[A-Za-z_][A-Za-z0-9_]*
61	Entero (decimal)	(variable)	TK_ENTERO	[0-9]+
62	Entero (hex)	(variable)	TK_ENTERO	0[xX][0-9A-Fa-f]+
63	Cadena y escape	(variable)	TK_CADENA	"([^"\\\n]|ESCAPE)*"
64	ESCAPE	—	—	\\[nt\\"]

Consideraciones

- TK_NOT nombra al ~ (NOT bit a bit) y TK_NON a la palabra reservada non (negación lógica). Son dos operadores distintos sobre tipos distintos: ~ opera sobre integer y non sobre veritas.

- Rango de los literales enteros. Un literal decimal debe de caber en el rango con signo, de 0 a $2^{31} - 1$; fuera de ahí es E106. Un literal hexadecimal se interpreta como un patrón de 32 bits, con rango $0x0$ a $0xFFFFFFFF$, y su valor como integer es el complemento a dos de ese patrón. Así $0xDEADBEEF$ es válido y vale -559038737. La asimetría es deliberada, las máscaras de bits y las subllaves del algoritmo de cifrado se escriben como patrones, no como numéricos con signo.
- La expresión regular de la fila 63 es la especificación de que es una cadena válida. En la implementación, las cadenas se reconocen con un estado exclusivo del leer, igual que los comentarios de bloque, porque así se puede reportar el escape inválido (E104) señalando el carácter exacto en lugar de rechazar la cadena entera.

c. Reglas que no producen token

Espacio en blanco	[\t\r\n]+	Se descarta
Comentario de línea	#[^{\n][^\n]*	Se descarta
Comentario de bloque	#\{ ... \}#	Se descarta
Fin de archivo	(marca virtual)	TK_EOF

El comentario de línea excluye explícitamente { como primer carácter. Sin esa exclusión, ante la cadena de entrada #{ texto }# la regla del comentario de línea consumirá toda la línea (que es una coincidencia más larga) y se tragaría el comentario de bloque. Es un pequeño detalle que rompe el lexer si se omite.

El comentario de bloque no se escribe como una sola expresión regular sino como un estado exclusivo del lexer: al ver #{ se entra al estado, se descarta todo hasta }# y se vuelve al estado inicial. Si llega el fin de archivo dentro del estado, es E103.

d. Desambiguación: el bocado más largo

Varios lexemas son prefijo de otro: > de >=, >> y >>>; < de <= y <<; - de -= y ->; = de ==. La regla es siempre la coincidencia más larga, pero quien la garantiza depende de la herramienta:

- Con flex, es automático: El generador elige la coincidencia más larga sin importar el orden en que escriben las reglas.
- Con una alternancia de expresiones (por ejemplo, modulo re de Python), el orden sí importa: gana la primera que coincida, así que hay que listar >>> antes que >>, >> antes que >= y >; << antes que <= y <; -> antes que -= y -; y == antes que =.

En la implementación entregada se usa flex, pero las reglas se escribieron igualmente de mayor a menor longitud para que la tabla sirva en ambos casos.

e. Errores que detecta el analizador léxico

Código	Situación	Cómo lo detecta
E101	Carácter no reconocido	Regla comodín al final de la tabla
E102	Cadena sin cerrar	Patrón de cadena sin comilla de cierre
E103	Comentario de bloque sin cerrar	Fin de archivo dentro del estado de comentario
E104	Escape inválido	Cadena bien delimitada cuyo contenido tiene una barra invertida seguida de un carácter que no es n, t, \ ni comilla

E105	Número mal formado	0[xX] sin dígitos; [0-9]+[A-Za-z_]+ (entero pegado a un identificador, por ejemplo 12a); 0[xX][0-9A-Fa-f]+[G-Zg-z_]+ (hexadecimal pegado a un identificador, por ejemplo 0x1FG
E106	Entero fuera de rango	Conversión en strtoll y comparación contra $2^{31} - 1$
E107	Identificador demasiado largo	Longitud del lexema mayor que 255
E208	Palabra reservada futura	Tabla de palabras reservadas para versiones futuras

Cada token lleva lexema, línea y columna; la línea la mantiene flex con yylineno y la columna se lleva a mano sumando la longitud de cada lexema y reiniciando en cada salto de línea. Son los datos que después aparecen en todos los mensajes de error.

18. Gramáticas Libre de Contexto (notación Bison)

Notación: MAYÚSCULAS = terminal (token que entrega el lexer); minúsculas = no terminal; 'x' = terminal literal de un solo carácter; : separa la cabeza del cuerpo; | = ó; %empty = producción vacía. Símbolo inicial: programa

Los operadores de más de un carácter (->, ==, <=, <<, +=, …) no pueden escribirse entre comillas simples porque Bison solo admite un carácter ahí. Van con el nombre de token que les da la tabla léxica: TK_FLECHA, TK_IGUAL, TK_MENOR_IGUAL, TK_DESP_IZQ, TK_ASIGNA_MAS, y así.

a. Declaración de terminales

%token <texto>	TK_IDENT TK_CADENA
%token <entero>	TK_ENTERO
%token	TK_FUNCTIO TK_VARIA TK_CONSTANS TK_PRIVATUM TK_IMPORTA TK_UT TK_REDDE
%token	TK_SI TK_SIN TK_ALITER TK_DUM TK_PER TK_RUMPE TK_PERGE
%token	TK_TIPO_INTEGER TK_TIPO_VERITAS TK_TIPO_TEXTUS TK_TIPO_NIHIL
%token	TK_VERUM TK_FALSUM
%token	TK_PAR_IZQ TK_PAR_DER TK_LLAVE_IZQ TK_LLAVE_DER
%token	TK_COR_IZQ TK_COR_DER TK_PUNTO_COMA TK_COMA
%token	TK_DOS_PUNTOS TK_PUNTO TK_FLECHA TK_ARROBA
%token	TK_ASIGNA TK_ASIGNA_MAS TK_ASIGNA_MENOS TK_ASIGNA_POR
%token	TK_ASIGNA_ENTRE TK_ASIGNA_MODULO

Todos los terminales se declaran con nombre, incluidos los de un solo carácter. No se usan literales entre comillas simples en ninguna producción. La razón es la coherencia con la tabla léxica: ahí los 36 símbolos ya tienen un nombre TK_, y

escribir '(' en la gramática y TK_PAR_IZQ en el lexer obligaría al lector a mantener dos vocabularios para la misma cosa.

Los operadores que llevan precedencia (TK_MAS, TK_POR, TK_AUT, TK_BARRA, …) no se declaran con %token sino con %left o %precedence en la sección siguiente, que también los declara como terminales.

b. Precedencia y asociatividad:

La regla expresión es ambigua a propósito; estas declaraciones son las que la desambiguan. Bison las lee de menor a mayor precedencia, que es el orden inverso al de la tabla del primer avance.

%left	TK_AUT
%left	TK_ET
%left	TK_BARRA
%left	TK_XOR
%left	TK_AMPERSAND
%left	TK_IGUAL TK_DIFERENTE
%left	TK_MENOR TK_MENOR_IGUAL TK_MAYOR TK_MAYOR_IGUAL
%left	TK_DESP_IZQ TK_DESP_DER TK_DESP_LOG
%left	TK_MAS TK_MENOS
%left	TK_POR TK_ENTRE TK_MODULO
%precedence	TK_NON TK_NOT UMENOS

UMENOS es un token ficticio: no lo produce el lexer y existe únicamente para que la producción expresion : TK_MENOS expresion %prec UMENOS tome la

precedencia del menos unario y no la del binario. Si se usa %prec sin declararlo, Bison falla.

El último nivel usa %precedence y no %right porque los operadores unarios no se asocian entre sí: non non x se resuelve por anidamiento, no por asociatividad, y Bison advierte si se declara una asociatividad que nunca se consulta.

c. Producciones:

```
/* Regla principal: el programa puede tener imports y declaraciones globales. */
programa
        : importaciones_opt declaraciones_globales_opt
        ;

/* Importaciones */
importaciones_opt
        : %empty
        | importaciones_opt importacion
        ;

importacion
        : TK_IMPORTA TK_CADENA TK_PUNTO_COMA
        | TK_IMPORTA TK_CADENA TK_UT TK_IDENT TK_PUNTO_COMA
        ;

/* Declaraciones globales */
declaraciones_globales_opt
        : %empty
        | declaraciones_globales_opt declaracion_global
        ;
```

```
/* Las anotaciones solo preceden a funciones. 'varia' no es valida fuera de
 * una funcion (E320), pero se reconoce para dar un error claro y seguir.
 * 'error' es la recuperacion en modo panico: descarta tokens hasta el inicio
 * de la siguiente declaracion global. */
declaracion_global
        : anotaciones privatum_opt decl_func
        | privatum_opt decl_func
        | privatum_opt decl_const
        | decl_var
        | error
        ;

privatum_opt
        : %empty
        | TK_PRIVATUM
        ;

anotaciones
        : anotacion
        | anotaciones anotacion
        ;
anotacion
        : TK_ARROBA TK_IDENT
        | TK_ARROBA TK_IDENT TK_PAR_IZQ TK_CADENA TK_PAR_DER
        ;

decl_func
        : TK_FUNCTIO TK_IDENT TK_PAR_IZQ parametros_opt TK_PAR_DER
        TK_FLECHA tipo bloque
        ;
```

```
parametros_opt
    : %empty
    | parametros
    ;

parametros
        : parametro
        | parametros TK_COMA parametro
        ;

parametro
        : TK_IDENT TK_DOS_PUNTOS tipo_param
        ;

decl_const
        : TK_CONSTANS TK_IDENT TK_DOS_PUNTOS tipo TK_ASIGNA expresion
        fin_sent
        ;

/* Tipos y dimensiones */
tipo
        : TK_TIPO_INTEGER dims
        | TK_TIPO_VERITAS dims
        | TK_TIPO_TEXTUS dims
        | TK_TIPO_NIHIL dims
        ;

dims
        : %empty
        | TK_COR_IZQ expresion TK_COR_DER
```

```
        | TK_COR_IZQ expresion TK_COR_DER TK_COR_IZQ expresion
        TK_COR_DER
        ;

tipo_param
        : TK_TIPO_INTEGER dims_param
        | TK_TIPO_VERITAS dims_param
        | TK_TIPO_TEXTUS dims_param
        | TK_TIPO_NIHIL dims_param
        ;

/* Un parametro es o totalmente dimensionado (dims) o totalmente abierto:
 * no se pueden mezclar, y el maximo es de dos dimensiones. */
dims_param
        : dims
        | TK_COR_IZQ TK_COR_DER
        | TK_COR_IZQ TK_COR_DER TK_COR_IZQ TK_COR_DER
        ;

/* Bloques y sentencias */
bloque
        : TK_LLAVE_IZQ sentencias_opt TK_LLAVE_DER
        ;

sentencias_opt
        : %empty
        | sentencias_opt sentencia
        ;

sentencia
        : decl_var
```

```
        | decl_const
        | sent_asignacion
        | sent_si
        | sent_dum
        | sent_per
        | sent_redde
        | sent_rumpe
        | sent_perge
        | sent_llamada
        | bloque
        | error TK_PUNTO_COMA
        | error
        ;

/* Fin de sentencia. Si falta el ';' y lo que sigue empieza otra sentencia (o
 * cierra el bloque), se inserta virtualmente: se reporta E202 y el analisis
 * continua sin descartar nada (correccion a nivel de frase). */
fin_sent
        : TK_PUNTO_COMA
        | %empty
        ;

/* El ')' que cierra la condicion de si/sin/dum/per se corrige igual. */
cierra_par
        : TK_PAR_DER
        | %empty
        ;

decl_var
        : TK_VARIA TK_IDENT TK_DOS_PUNTOS tipo fin_sent
```

```
        | TK_VARIA TK_IDENT TK_DOS_PUNTOS tipo TK_ASIGNA inicializador
        fin_sent
        ;

inicializador
        : expresion
        | lista_valores
        ;

lista_valores
        : TK_LLAVE_IZQ TK_LLAVE_DER
        | TK_LLAVE_IZQ lista_inicializadores TK_LLAVE_DER
        ;

lista_inicializadores
        : inicializador
        | lista_inicializadores TK_COMA inicializador
        ;

sent_asignacion
        : asignacion fin_sent
        ;

/* Las asignaciones compuestas se expanden a la forma simple. */
asignacion
        : designador TK_ASIGNA expresion
        | designador TK_ASIGNA_MAS expresion
        | designador TK_ASIGNA_MENOS expresion
        | designador TK_ASIGNA_POR expresion
        | designador TK_ASIGNA_ENTRE expresion
        | designador TK_ASIGNA_MODULO expresion
```

```
        ;

/* Maximo dos indices, igual que el maximo de dimensiones. */
designador
        : TK_IDENT
        | TK_IDENT TK_COR_IZQ expresion TK_COR_DER
        | TK_IDENT TK_COR_IZQ expresion TK_COR_DER TK_COR_IZQ
        expresion TK_COR_DER
        ;

/* 'sin' se convierte en una cadena de ifs anidados. */
sent_si
        : TK_SI TK_PAR_IZQ expresion cierra_par bloque cola_si
        ;

cola_si
        : %empty
        | TK_ALITER bloque
        | TK_SIN TK_PAR_IZQ expresion cierra_par bloque cola_si
        ;

sent_dum
        : TK_DUM TK_PAR_IZQ expresion cierra_par bloque
        ;

sent_per
        : TK_PER TK_PAR_IZQ per_init TK_PUNTO_COMA expresion
        TK_PUNTO_COMA asignacion cierra_par bloque
        ;

per_init
```

```
        : TK_VARIA TK_IDENT TK_DOS_PUNTOS tipo TK_ASIGNA expresion
        | asignacion
        ;

sent_redde
        : TK_REDDE TK_PUNTO_COMA
        | TK_REDDE expresion fin_sent
        ;

sent_rumpe
        : TK_RUMPE fin_sent
        ;

sent_perge
        : TK_PERGE fin_sent
        ;

sent_llamada
        : llamada_expr fin_sent
        ;

llamada_expr
        : TK_IDENT TK_PAR_IZQ lista_argumentos TK_PAR_DER
        | TK_IDENT TK_PUNTO TK_IDENT TK_PAR_IZQ lista_argumentos
        TK_PAR_DER
        ;

lista_argumentos
        : %empty
        | argumentos
        ;
```

```
argumentos
        : expresion
        | argumentos TK_COMA expresion
        ;

/* Expresiones. La precedencia la resuelve Bison con las reglas %left/%right. */
expresion
        : expresion TK_AUT expresion
        | expresion TK_ET expresion
        | expresion TK_BARRA expresion
        | expresion TK_XOR expresion
        | expresion TK_AMPERSAND expresion
        | expresion TK_IGUAL expresion
        | expresion TK_DIFERENTE expresion
        | expresion TK_MENOR expresion
        | expresion TK_MENOR_IGUAL expresion
        | expresion TK_MAYOR expresion
        | expresion TK_MAYOR_IGUAL expresion
        | expresion TK_DESP_IZQ expresion
        | expresion TK_DESP_DER expresion
        | expresion TK_DESP_LOG expresion
        | expresion TK_MAS expresion
        | expresion TK_MENOS expresion
        | expresion TK_POR expresion
        | expresion TK_ENTRE expresion
        | expresion TK_MODULO expresion
        | TK_MENOS expresion %prec UMENOS
        | TK_NON expresion
        | TK_NOT expresion
        | TK_PAR_IZQ expresion TK_PAR_DER
```

```
        | referencia
        | TK_ENTERO
        | TK_CADENA
        | TK_VERUM
        | TK_FALSUM
        ;

/* Maximo dos indices. Una llamada no se puede indexar: las funciones no
 * retornan arreglos, asi que f(x)[0] no tendria sentido. */
referencia
        : nombre_ref
        | nombre_ref TK_COR_IZQ expresion TK_COR_DER
        | nombre_ref TK_COR_IZQ expresion TK_COR_DER TK_COR_IZQ
        expresion TK_COR_DER
        | llamada_expr
        ;

nombre_ref
        : TK_IDENT
        | TK_IDENT TK_PUNTO TK_IDENT
        ;
```

Consideraciones

- Tres reglas auxiliares no existen en la gramática formal de la sección 19 y son exclusivas de la implementación, porque solo sirven para dar mejores diagnósticos: fin_sent aplica la corrección a nivel de frase cuando falta un ;, cierra_par reporta el paréntesis faltante de una condición, y nombre_ref factoriza el nombre calificado modulo.simbolo para que referencia y llamada_expr no entren en conflicto.

- declaracion_global deriva decl_var a propósito. Una varia fuera de una función es sintácticamente reconocible, así que el parser la acepta y reporta E320 en la acción, en vez de dejar que el analizador semántico la detecte más tarde. Esto permite dar el mensaje exacto y seguir analizando el resto del archivo. Es la única regla del lenguaje que se valida en la fase 2 llevando un código de la familia E3xx.
- El no terminal dims de esta gramática no es el Dims → ε | Dims '[' Expresion ']' que se descartó. Aquí enumera exactamente tres alternativas (vacío, una dimensión y dos) de modo que el límite sigue impuesto por la gramática. Se factorizó en un no terminal aparte para no repetir las tres formas en las cuatro alternativas de tipo y en las cuatro de tipo_param.

19. Gramática equivalente en notación estándar:

Programa	→ Importaciones Declaraciones
Importaciones	→ ε | Importaciones Importacion
Importacion	→ importa cadena ';'
	| importa cadena ut ident ';'
Declaraciones	→ ε | Declaraciones Declaracion_global
Declaracion_global	→ Anotaciones Privatum Decl_func
	| Privatum Decl_func
	| Privatum Decl_const
Privatum	→ ε | privatum
Anotaciones	→ Anotacion | Anotaciones Anotacion
Anotacion	→ '@' ident | '@' ident '(' cadena ')'

Decl_func → functio ident '(' Parametros_opt ')' '->' Tipo Bloque

Parametros_opt → ε | Parametros

Parametros → Parametro | Parametros ',' Parametro

Parametro → ident ':' Tipo_param

Tipo_base → integer | veritas | textus | nihil

Tipo → Tipo_base Dims

Dims → ε

| '[' Expresion ']'

| '[' Expresion ']' '[' Expresion ']'

Tipo_param → Tipo_base Dims_param

Dims_param → Dims

| '[' ']'

| '[' ']' '[' ']'

Bloque → '{' Sentencias '}'

Sentencias → ε | Sentencias Sentencia

Sentencia → Decl_var | Decl_const | Sent_asignacion | Sent_si
| Sent_dum | Sent_per | Sent_redde | Sent_rumpe

| Sent_perge | Sent_llamada | Bloque

Decl_var → varia ident ':' Tipo ';'

| varia ident ':' Tipo '=' Inicializador ';'

Decl_const → constans ident ':' Tipo '=' Expresion ';'

Inicializador → Expresion

| Lista_valores

Lista_valores → '{' '}' | '{' Lista_inicializadores '}'

Lista_inicializadores → Inicializador | Lista_inicializadores ',' Inicializador

Sent_asignacion → Asignacion ';'

Asignacion → Designador '=' Expresion

| Designador '+=' Expresion

| Designador '-=' Expresion

| Designador '*=' Expresion

| Designador '/=' Expresion

| Designador '%=' Expresion

Designador → ident

| ident '[' Expresion ']'

| ident '[' Expresion ']' '[' Expresion ']'

Sent_si → si '(' Expresion ')' Bloque Cola_si

Cola_si → ε

| aliter Bloque

| sin '(' Expresion ')' Bloque Cola_si

Sent_dum → dum '(' Expresion ')' Bloque

Sent_per → per '(' Per_init ';' Expresion ';' Asignacion ')' Bloque

Per_init → varia ident ':' Tipo '=' Expresion | Asignacion

Sent_redde → redde ';' | redde Expresion ';'

Sent_rumpe → rumpe ';'

Sent_perge → perge ';'

Sent_llamada → Llamada ';'

Llamada → Nombre '(' Argumentos_opt ')'

Nombre → ident | ident '.' ident

Argumentos_opt → ε | Argumentos

Argumentos → Expresion | Argumentos ',' Expresion

Referencia → Nombre

| Nombre '[' Expresion ']'

| Nombre '[' Expresion ']' '[' Expresion ']'

| Llamada

Expresion → Expresion aut Expresion | Expresion et Expresion

| Expresion '|' Expresion | Expresion '^' Expresion

| Expresion '&' Expresion | Expresion '==' Expresion

| Expresion '!=' Expresion | Expresion '<' Expresion

| Expresion '<=' Expresion | Expresion '>' Expresion

| Expresion '>=' Expresion | Expresion '<<' Expresion

| Expresion '>>' Expresion | Expresion '>>>' Expresion

| Expresion '+' Expresion | Expresion '-' Expresion

| Expresion '*' Expresion | Expresion '/' Expresion

| Expresion '%' Expresion | '-' Expresion | non Expresion

| '~' Expresion | '(' Expresion ')'

| Referencia | entero | cadena | verum | falsum

Consideraciones

- La ambigüedad de Expresion se resuelve con la tabla de precedencia de la sección 18.b. Si se quisiera una gramática no ambigua sin la tabla de precedencia, es la versión LL(1) de doce niveles encadenados del primer avance.
- La gramática acepta la lista vacía {} por simetría, pero ningún tipo la puede satisfacer: toda dimensión declarada es mayor que cero. El caso se rechaza en el análisis semántico con E317, junto con el resto de las listas cuya forma no coincide con el tipo.
- Cola_si es recursiva por la derecha, y por eso una cadena de sin se pliega naturalmente en los si anidados equivalentes: cada sin queda como el bloque alternativo del anterior. Es el mismo plegado que documenta la sección 1 del primer avance, y se puede verificar imprimiendo el AST con -t.

20. Arreglos y matrices

Esta sección corresponde al comentario recibido en la revisión del primer avance. El cambio de fondo es que el límite de dos dimensiones pasó de ser una regla semántica a ser una regla de la gramática. Antes, $Dims \rightarrow \varepsilon \mid Dims\ '['\ Expresion\ ']'$ aceptaba cualquier cantidad de corchetes y había que rechazar la tercera dimensión en el analizador semántico. Ahora, la gramática simplemente no la deriva, y $integer[2][2][2]$ es un error de sintaxis.

Las cuatro reglas quedan así:

Construcción	Forma	Qué acepta
Tipo de variable o de retorno	Tipo	integer, integer[N], integer[N][M]
Tipo de parámetro	Tipo_param	Lo anterior, más integer[] e integer[][]
Lado izquierdo de una asignación	Designador	a, a[i], a[i][j]
Uso en una expresión	Referencia	a, mod.a, a[i], a[i][j], y llamadas

Tres consecuencias que conviene mencionar:

1. Las dimensiones abiertas no se mezclan con las dimensionadas. Tipo_param ofrece o la forma completamente dimensionada (Tipo) o la completamente abierta ([], [][]). Un parámetro como integer[3][] ya no existe, lo que elimina el caso raro de una función que fija una dimensión y otra la deja libre.
2. No se puede indexar el resultado de una llamada. Referencia deriva Llamada, pero no Llamada '[' Expresion ']'. Es coherente con el lenguaje: las funciones no pueden retornar arreglos, así que $f(x)[0]$ nunca habría tenido sentido. El caso está cubierto por test/errores/E201_indexar_llamada.nv.
3. El triple índice es un error de sintaxis. $m[i][j][k]$ se rechaza en el parser, con línea y columna, en vez de llegar al análisis semántico.

Retorno de una posición. El ejercicio de búsqueda en matriz pide retornar la posición del elemento, que son dos valores. Como una función de Nova no puede retornar un arreglo, el programa de ejemplo lo resuelve con un parámetro de salida: buscar(m : integer[][], objetivo : integer, pos : integer[]) -> veritas escribe la fila en pos[0] y la columna en pos[1], y devuelve veritas indicando si lo encontró. Es la consecuencia directa de que los arreglos viajen por referencia y no puedan retornarse.

Lo que sigue siendo responsabilidad del analizador semántico, porque una gramática libre de contexto no puede expresarlo:

- Que las dimensiones declaradas sean expresiones constantes positivas (E318).
- Que la lista de inicialización tenga exactamente la forma del tipo declarado (E317): la gramática acepta cualquier anidamiento de llaves.
- Que un índice constante esté dentro del rango (E308) y que el índice sea integer (E307).
- Que el tipo de elemento y la cantidad de dimensiones coincidan al pasar un arreglo a una función (E314).
- Que nihil no se use como tipo de un arreglo ni de una variable (E321): la gramática lo deriva por simplicidad, igual que en el primer avance.

21. Análisis semántico y tabla de símbolos

La fase 3 recibe el árbol sintáctico y devuelve dos cosas: el mismo árbol con el tipo resuelto en cada expresión y la dirección de memoria en cada nombre (el árbol decorado), y la tabla de símbolos. Se inspecciona con $novac --sym$.

a. Estructura

La tabla es una pila de ámbitos. El nivel 0 es el ámbito global y contiene los módulos importados, las constantes, las funciones del archivo y los siete identificadores predefinidos (longitudo, scribe, dic, ingredere, exi, custodi, feistel). Cada función abre el nivel 1 con sus parámetros y cada bloque abre uno más. La búsqueda recorre la pila de adentro hacia afuera y se detiene en la primera coincidencia: eso es lo que hace que ocultar un nombre sea legal pero avisado (W002), mientras

declararlo dos veces en el mismo ámbito es E302. Al cerrarse ese ámbito se emite W001 por lo que se declaró y nunca se leyó.

Cada símbolo guarda el nombre y la clase (varia, parámetro, constans, functio, módulo), el tipo, las dimensiones (0, 1 o 2, con filas y columnas cuando se conocen o marcadas como abiertas), la dirección y el tamaño en bytes, el valor plegado si es una constante evaluable, la firma completa si es una función, el tamaño del marco, y la línea y la columna de la declaración para los diagnósticos.

b. Dos pasadas

El nivel global se recorre dos veces antes de entrar a los cuerpos: primero las constantes, después las firmas de las funciones. Son dos barridos y no uno porque un parámetro puede declararse como integer[N][N] usando una constante, de modo que las constantes tienen que existir antes que las firmas. Con todas las firmas registradas, el orden en que se escriban las funciones dentro del archivo deja de importar. La segunda pasada entra a los cuerpos y verifica tipos, ámbitos, dimensiones y flujo.

El plegado de constantes ocurre en esta fase porque el compilador ya debe evaluar expresiones constantes para conocer el tamaño de los arreglos. El mismo evaluador es el que detecta el índice constante fuera de rango (E308), la división entre la constante cero (E316) y la ranura o ronda fuera del rango de la bóveda (E319).

c. Asignación de memoria

Las direcciones se reparten sobre el mapa de datos del procesador CE 4301. Una palabra son 4 bytes y todo queda alineado a palabra.

Zona	Rango	Quién lo administra
Datos globales	$0x0000 - 0x3FFF$ (16 KB)	Este compilador
Buffers de archivos	$0x4000 - 0xBFFF$	El programa de cifrado
Heap reservado	$0xC000 - 0xEFFF$	Sin uso en esta versión
Pila	$0xF000 - 0xFFFC$	El generador, con R14

Una constans no ocupa espacio: su valor se incrusta como un inmediato en las instrucciones que la usan. Un escalar ocupa una palabra; un arreglo de N elementos ocupa $4N$; una matriz de $N \times M$ ocupa $4NM$. Un arreglo viaja a una función por referencia, como un descriptor de dirección base más un tamaño por dimensión: 8 bytes si es [] y 12 si es [][]. La tabla registra el desplazamiento del dato dentro del área de locales, y la fase 4 lo convierte en el desplazamiento definitivo respecto de R14 cuando ya conoce el tamaño completo del marco.

Como Nova solo admite declaraciones globales constans, en la práctica no hay datos globales mutables y todas las variables viven en la pila. W007 avisa cuando un marco o los datos globales no caben en el espacio que el ISA les da.

d. Qué verifica

Grupo	Códigos
Nombres y ámbitos	E301 no declarado, E302 redeclaración, E311 asignación a una constante, E313 no es una función
Tipos	E303 tipos incompatibles, E306 condición no booleana, E312 tipo de retorno, E321 nihil fuera de retorno

Llamadas	E304 cantidad de argumentos, E305 tipo de argumento, E319 argumento de intrínseca
Arreglos	E307 índice no entero, E308 índice constante fuera de rango, E314 dimensiones incompatibles, E317 forma del inicializador, E318 dimensión no constante
Flujo	E309 falta redde, E310 rumpe fuera de ciclo, E315 @cauda incumplida
Constantes	E316 división entre cero
Módulos	E402 espacio de nombres duplicado, E409 nombre de módulo inválido
Advertencias	W001 declarado sin usar, W002 oculta otra declaración, W003 código inalcanzable, W004 anotación desconocida, W005 valor de retorno descartado, W006 función @obsoletum, W007 no cabe en memoria

E320 (declaración global que no es constans), no está en esta lista porque lo rechaza la gramática, con una producción dedicada que da el mensaje y sigue analizando, igual que el límite de dos dimensiones de la sección 20.

Dos verificaciones merecen una nota porque no son comprobaciones locales. E309 es un análisis real de flujo: una función con tipo de retorno debe retornar por todos los caminos, y un dum (verum) sin rumpe cuenta como camino que no se cae por el final. E315 verifica que en una función marcada @cauda toda llamada recursiva esté en posición de cola, lo que es lo que habilita la optimización de la fase 4.

22. Generación de código

La fase 4 recibe el árbol decorado y la tabla de símbolos, y produce el ejecutable de CERBERO. Lo hace en dos pasos inspeccionables por separado: novac –ir imprime el código de tres direcciones y novac -s el ensamblador VLIW.

a. Código de tres direcciones

Cada operación queda como destino = operando1 op operando2, con temporales t0, t1, … y etiquetas L0, L1, …. Las estructuras de control se aplanan a saltos condicionales e incondicionales; et y aut se traducen con saltos para que evalúen en corto circuito; los accesos a arreglo se linealizan por filas (m[i][j] es i*columnas + j) y arrastran su verificación de rango. Si el arreglo tiene dimensiones conocidas el paso es una constante, y si es un parámetro de dimensiones abiertas se lee del descriptor, que es lo mismo que consulta longitudo. Las llamadas quedan como una secuencia de param seguida de call.

b. Selección de instrucciones

La traducción al repertorio de CERBERO es casi siempre directa; la tabla resume los casos en que el ISA obliga a resolverlo de otra manera:

Operación de Nova	Cómo se emite
+ - * & | ^ << >> >>>	add sub mul and or xor sll sra srl, una instrucción
- unario, ~	neg y not, una instrucción

== != < >	cmp más un salto condicional; el resultado se materializa en 0 o 1
<= >=	el ISA no tiene LE ni GT: se intercambian los operandos y se usa GE o LT
non	cmpi a, #0 más un salto: se materializa como a == 0
/ %	el ISA no tiene DIV: se llama a __div32, restas desplazadas en 32 iteraciones
scribe dic	escrituras en una zona de salida en memoria, porque el ISA no define puertos de E/S
ingredere custodi feistel	authw+login, ksetw y f4e, en el slot S3

c. Marco de pila y convenciones de llamada

Los cuatro primeros argumentos van en R6–R9 y el resto en el área baja del marco del llamador; el valor de retorno vuelve en R6 y la dirección de retorno en R15. De la dirección que apunta R14 hacia arriba:

[0 .. 4k-4]	argumentos salientes a partir del quinto
[4k .. +L-4]	parámetros y locales (los ubicó la fase 3)
[4k+L .. +4T-4]	temporales del código de tres direcciones
[total-4]	R15 guardado, solo si la función llama a otra

R14 no se mueve dentro del cuerpo, así que todos los desplazamientos son constantes y los argumentos por pila quedan exactamente donde el llamado los espera; el área de argumentos salientes va al fondo justamente por eso. R10 y R11 son los temporales del generador, y R0:R1 se reservan como el par criptográfico P0 de Feistel4, con lo que se respeta la tabla de convenciones del ISA sin tocarla.

No hay asignación de registros con análisis de vivacidad: cada valor vive en memoria y cada operación carga sus operandos, calcula y guarda. Con dos temporales libres es el modelo que siempre funciona. Lo único que se agregó es el reuso del último valor dejado en un registro, que se invalida en cada etiqueta, salto y llamada.

d. Las dos reglas temporales del ISA

Entre quien produce un valor y quien lo consume tienen que pasar tres bundles, y después de cada salto hay un delay slot que se ejecuta siempre. El generador lleva la cuenta de en qué bundle queda disponible cada registro y las banderas, e inserta bundles vacíos solo cuando hacen falta. Antes de cada salto y de cada etiqueta pone una barrera, para que ningún valor a medio camino cruce un borde del flujo de control.

e. Errores de ejecución

La verificación de rango se emite en cada acceso a arreglo y la comparación es sin signo, lo que atrapa también los índices negativos: como entero sin signo quedan por encima del límite. Los tres códigos que el lenguaje fija se producen dejando el valor en R6 antes del halt: 134 índice fuera de rango, 136 división entre cero (la

comprobación está al entrar a __div32) y 137 uso de la bóveda sin autenticar, con una guarda br.auth antes de custodi y de feistel.

f. Recursividad de cola

En una función marcada @cauda, una llamada recursiva no apila un marco nuevo: los argumentos se reasignan a los parámetros y el control vuelve a una etiqueta puesta justo después del prólogo. El retorno que sí termina la recursión devuelve su valor al llamador original, porque el marco nunca se volvió a apilar. Se mide: una función que suma 1 a 500 por recursión de cola usa 48 bytes de pila, y la misma función sin la anotación usa 18 056 bytes y desborda los 4 KB que el ISA le da a la pila.

g. Relajación de saltos

El desplazamiento del formato tipo C son 10 bits contados en bundles, o sea ±512, y emitiendo una instrucción por bundle un programa mediano se pasa de ahí. Para esos casos el ISA tiene el tipo J, que salta a la dirección que trae un registro:

```
movi r11, #&destino        ; dirección de la etiqueta, en bytes
jalr.al r11                ; o jr.al r11 si era un salto y no una llamada
```

Cuáles sitios la necesitan no se sabe hasta conocer las posiciones finales, y la forma larga ocupa más bundles, así que mueve las posiciones. Por eso el programa se construye, se miran los saltos que quedaron fuera de rango, se marcan y se vuelve

a construir, hasta que ninguno se desborda. Solo se relaja lo necesario: factorial.nv no usa ninguno, mergesort.nv usa 8 y multiplicacion.nv 11.

h. El ejecutable

Sin banderas de volcado, novac produce el ejecutable. Cada bundle son 16 bytes en little-endian, con los cinco slots en los campos del ISA: S0 en 127:96, S1 en 95:64, S2 en 63:32, S3 en 31:16 y S4 en 15:0. Están implementados los ocho formatos: A (rd, rs1, rs2), I (rd, rs1, imm19), M (rd, #desp(rbase)), F (Feistel), V (bóveda), AC (ALU corta), C (salto relativo en bundles) y J (salto por registro). Un detalle del repertorio: movi y movh existen para armar entre los dos un patrón de 32 bits, así que su inmediato se toma como los 19 bits completos sin signo; los demás inmediatos sí son con signo, porque se suman o se comparan.

Ni la definición de Nova ni la especificación del ISA fijan un formato de archivo, así que el siguiente es el que propone el compilador y hay que acordarlo con el cargador de CE4301. Son 48 bytes de encabezado:

Desp.	Campo	Contenido
0	magia	"NOVA"
4	versión	u16 versión, u16 reservado
8	n_bundles	bundles de 16 bytes que siguen al encabezado
12	codigo_org	dirección de IMEM donde se carga el código
16	entrada	dirección de la primera instrucción

20	datos_org	dirección de DMEM donde empieza la sección de datos
24	datos_init	bytes inicializados que trae el archivo
28	datos_cero	bytes que el cargador pone en cero a continuación
32	pila_tope	valor inicial de R14 (0xFFFC)
36	suma	verificación de código más datos inicializados
40	reservado	dos palabras para el enlazador

Los datos se parten en dos porque la zona de salida son 8 KB que empiezan en cero: el archivo trae solo los literales de cadena y le dice al cargador cuántos bytes limpiar después. El ensamblador en texto y el ejecutable binario se construyen desde la misma lista de bundles, así que las dos salidas no pueden divergir.

i. Entrada y salida

El ISA no define puertos de entrada y salida: el rango $0x4000 - 0xBFFF$ son buffers de archivos a cifrar, no una consola. Siguiendo lo que la definición del lenguaje prevé para este caso, scribe y dic se compilan a escrituras sobre una zona de salida en memoria, dentro del segmento de globales que el compilador administra: $0x2000$ en adelante los literales de cadena, $0x3000$ la cantidad de palabras escritas y $0x3004$ en adelante la zona que leen las

herramientas de CE4301. Un textus se guarda como una palabra de longitud seguida de sus bytes; dic agrega un $0x0A$ de fin de línea.

j. Estado

De la familia E4xx están implementados E401, E402, E405 y E409. Los cinco restantes (E403 importación circular, E404 módulo no encontrado, E406 principale duplicada, E407 firma incompatible entre objetos y E408 símbolo privado) corresponden al enlazador.

E410, verificación interna del generador. Avisa que la fase 4 no encontró algo que la fase 3 debió dejar ubicado, o que no pudo codificar una instrucción. Por construcción no se puede provocar desde un fuente válido: si aparece, es un error del compilador y no del programa que se compila. Es el único código sin archivo de prueba.

Falta la planificación, que es la fase 5: hoy se emite una instrucción por bundle, con una ocupación medida de 0,22 a 0,49 instrucciones por bundle y los slots S1 y S3 sin usar. La ALU corta de S3 existe justamente para esto. Falta también el enlazador: los programas de un solo archivo están completos, y los que importan otro módulo generan la llamada con el nombre decorado modulo$simbolo y se reportan con E401. Las banderas -c e -l están reconocidas y avisan que corresponden al enlazador, en lugar de dejar un archivo que parezca un objeto sin serlo.

23. Implementación y validación

a. Construcción

El proyecto está organizado en fase1_lexico/ (nova.l, diag.c, diag.h), fase2_sintactico/ (nova.y, ast.c, ast.h), fase3_semantico/ (symtab.c, symtab.h, semantic.c, semantic.h), fase4_codigo/ (tac.c, tac.h, codegen.c, codegen.h), tools/, test/ y main.c. El módulo diag concentra el reporte de errores de todas las fases, de modo que el formato de los diagnósticos se define en un solo lugar.

b. Resultados

Se analizaron los ocho archivos de ejemplo del primer avance, sin modificarlos, más cuatro programas nuevos: desplazamientos.nv para los tres desplazamientos, saludo.nv para las cadenas, traza.nv para el recorrido de una matriz y operadores.nv para los operadores compuestos y la importación con alias:

Archivo	Resultado
test/factorial.nv	Compila y genera ejecutable
test/ejemplos/multiplicacion.nv	Compila y genera ejecutable
test/ejemplos/mergesort.nv	Compila y genera ejecutable
test/ejemplos/matriz.nv	Módulo de biblioteca
test/ejemplos/estadistica.nv	Módulo de biblioteca
test/ejemplos/convolucion.nv	Importa otro módulo
test/ejemplos/bits_y_hex.nv	Compila y genera ejecutable
test/ejemplos/busqueda_matriz.nv	Compila y genera ejecutable
test/ejemplos/desplazamientos.nv	Compila y genera ejecutable
test/ejemplos/operadores.nv	Importa otro módulo
test/ejemplos/saludo.nv	Compila y genera ejecutable
test/ejemplos/traza.nv	Compila y genera ejecutable

Los cuatro últimos llegan hasta el ensamblador y se detienen con E405 (no hay principale) o E401 (símbolo de otro módulo), que es la familia que resuelve el enlazador.

A estos doce se suman los cuatro de test/ejecucion/ (rango.nv, div_cero.nv, boveda.nv y cola.nv), que no están en esta tabla porque lo que verifican no es que compilen sino con qué código terminan al ejecutarse.

c. Suite de pruebas

test/run_tests.sh automatiza la validación y se corre con make prueba. Cubre cinco cosas:

1. Los 16 archivos válidos deben analizarse sin errores, y además terminar sin errores con --tokens y con -t.
2. Cada archivo de test/errores/ debe fallar con el código que lleva en el nombre: hay un caso por cada error léxico (E101 a E107), cada error sintáctico (E201 a E209), errores semánticos (E301 a E321) y los de enlace E401, E402, E405 y E409. A su vez, cada archivo de test/advertencias/ cubre W001 a W007.
3. Los archivos multi_*.nv verifican que se reporten varios errores en una sola corrida y en el orden correcto; la primera línea del archivo declara los códigos esperados.
4. Las opciones --max-errors, --tokens y los códigos de salida 0, 1, 2 y 3.
5. Cada ejemplo completo produce su ejecutable y que el ejecutable binario da el mismo resultado que su ensamblador al ejecutarlo.

Resultado actual: 152 pruebas, 0 fallidas.

Ejemplo de salida con errores reales:

```
test/errores/multi_mezcla.nv:3:28: error[E201]: se esperaba ';' pero se encontro '['
3 |     varia a : integer[2][2][2];
  |                           ^
test/errores/multi_mezcla.nv:4:27: error[E101]: caracter no valido '$' fuera de una cadena
4 |     varia x : integer = 1 $ 2
  |                         ^
test/errores/multi_mezcla.nv:4:29: error[E201]: se esperaba ';' pero se encontro '2'
4 |     varia x : integer = 1 $ 2
  |                           ^

compilacion abortada: 6 errores (2 lexicos, 4 sintacticos).
```

El formato con la línea fuente y la marca ^ bajo el lexema culpable es el que especifica la sección de diagnósticos del primer avance, y lo produce diag.c para todas las fases. El resumen final desglosa por familia de código.

La recuperación va más allá del modo pánico básico: tras un E203 (falta un }) se suprimen los errores sintácticos posteriores, porque a partir de ahí el archivo ya no tiene estructura confiable y los mensajes serían ruido en cascada; y un token que ya provocó un error léxico se marca para no generar además un E202 de rebote.

d. Verificación por fases

depurar.sh corre 165 verificaciones agrupadas por fase, con make depurar F=todo o una fase a la vez. Además de revisar cada fase por separado, ejecuta el ensamblador generado con tools/simular.py, que modela la latencia de tres bundles del ISA: un valor escrito en el bundle N recién se puede leer en el N+3. Si el relleno estuviera mal, estos resultados saldrían distintos.

Programa	Resultado al ejecutarlo
factorial.nv	factorial(5) da 120
traza.nv	la traza de la matriz da 15
multiplicacion.nv	el producto da [[58,64],[139,154]]
mergesort.nv	[1, 3, 9, 10, 27, 38, 43, 82], ordenado
bits_y_hex.nv	−65419, con 0xDEADBEEF armado con movi+movh
desplazamientos.nv	[3840, 15, −16, 268435440]
saludo.nv	"Salve, munde" llega entera a la zona de salida
rango.nv, div_cero.nv, boveda.nv	terminan con 134, 136 y 137
cola.nv	la recursión de cola suma 1..500 con 48 bytes de pila

El simulador también carga el ejecutable binario y lo decodifica slot por slot: los doce programas completos dan el mismo resultado ejecutados como ensamblador y como binario, que es la prueba de que la codificación de 128 bits está bien armada.

24. Resumen del avance y nota sobre la clase de gramática

Se especifican las cuatro piezas que convierten la definición de sintaxis del avance anterior en algo analizable por una herramienta:

1. El analizador léxico, como un conjunto de expresiones regulares que cubre las 23 palabras reservadas, los 36 símbolos, los literales y los comentarios del lenguaje.
2. La gramática libre de contexto de todo el lenguaje, en dos notaciones: la de Bison (que es la que se implementa) y la notación estándar de gramáticas, equivalente.
3. El analizador semántico, con la tabla de símbolos, la verificación de tipos, ámbitos, dimensiones y flujo, y la asignación de memoria sobre el mapa de datos de CERBERO.
4. El generador de código, que produce el código de tres direcciones, el ensamblador VLIW y el ejecutable binario, verificados ejecutándolos sobre un simulador que modela la latencia del ISA.

Las cuatro están implementadas y probadas: las dos primeras con flex y bison, y las dos últimas en C; la sección 23 muestra la validación sobre los programas de ejemplo del avance anterior.

a. Nota sobre la gramática

El documento del primer avance presenta la gramática en EBNF y afirma que es LL(1), implementable con un analizador descendente recursivo. Esa sigue siendo la especificación de referencia del lenguaje, porque es la forma más legible de documentar la precedencia de los operadores.

La gramática de este avance es su equivalente LALR(1), que es lo que espera un generador como Bison. Las dos diferencias son deliberadas.

- Usa recursión por la izquierda (parametros : parametros ',' parametro), que un descendente recursivo no admite pero que es la forma natural y más compacta para un analizador ascendente.
- La regla expresion se escribe ambigua a propósito, en una sola producción por operador, y la ambigüedad se resuelve con declaraciones de precedencia. Esto sustituye los doce niveles encadenados de la versión LL(1), con el mismo resultado: $a + b * c$ se agrupa como $a + (b * c)$ en ambas.

Las dos gramáticas generan el mismo lenguaje. La LL(1) documenta, la LALR(1) se compila.